CHAPTER 16

Classification and Inference

16.0 Introduction

This chapter groups together a selection of computational techniques whose common feature is that they treat problems of classification and inference on complex models. Given substantially more space, the chapter might have been expanded to be a more complete survey of *machine learning*; but at its present length, it cannot pretend to be such. (A few general references are given below.)

Classification and inference, in a loose sense, are also the goals of many of the purely statistical methods that we already discussed in Chapters 14 and 15, and the line between such techniques and machine learning is a fuzzy one. This chapter's topics tend to have one or both of two characteristics: the underlying model (i) has discrete or combinatorial aspects that distinguish it from "classical" statistical methods, and/or (ii) has empirical or heuristic aspects that make exact statistical treatments unattainable.

There is a list of important topics, related to those in this chapter, that we would have wanted to include if only they could have been reduced to suitable length. *Bayesian networks* is at the top of this list. Section 16.0.1 gives an example of the kind of problem that a Bayesian network can solve. Other significant topics that we must omit include

- genetic algorithms
- neural nets
- kernel methods more general than those discussed in §16.5

16.0.1 Bayesian networks

These are sometimes called *Bayes nets*, *Bayesian learning networks*, or *belief networks*. Here we want only to give a flavor of the method, so that you will know when to consult the references below.

A Bayesian network consists of nodes, each of which can have a value. The values can be {true,false}, or a set of possibilities like {low, medium, high}, or an integer. Figure 16.0.1 shows an example where all the nodes have true/false values.

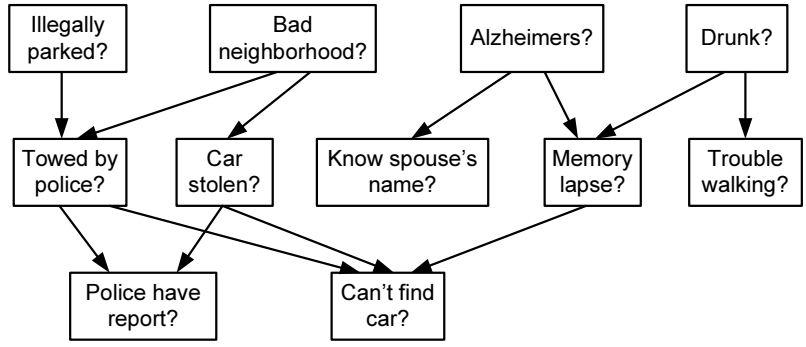


Figure 16.0.1. Example of a Bayesian network. Evidence about any node can be propagated to give probabilistic conclusions about any other node.

Each node in a network has a set of *prior probabilities* or *priors* that give the likelihood of its values absent any additional evidence. If a node has one or more *parents*, then its priors are conditioned on the values of the parents. For example, referring to the Figure, we might have

P(Illegally-Parked = true)
0.20

Bad-N'hood	P(Car-Stolen = true | Bad-N'hood)
true	0.05
false	0.001

Alzheimers	Drunk	P(Memory-Lapse = true | Alzheimers, Drunk)
true	true	0.999
true	false	0.95
false	true	0.50
false	false	0.01

And so on, for all the other nodes.

Things get interesting when we have some evidence to assimilate into the network. For example, you might be coming out of a bar in a bad neighborhood, walking with some difficulty, and be unable to find your car. Is it stolen? The Bayesian network theory gives algorithms for propagating information both up (from "Can't find car?") and down (from "Bad neighborhood?") to get new *posterior* estimates for the probabilities at all nodes, including, here, "Car stolen?" You can also compute in advance the value of new evidence. For example, how much would it help to call the police and see if there is a police report of a towed or recovered, stolen, car?

For more than this brief taste, see [1-3].

CITED REFERENCES AND FURTHER READING:

Hastie, T., Tibshirani, R., and Friedman, J.H. 2003, *The Elements of Statistical Learning* (Berlin: Springer).

Duda, R.O., Hart, P.E., and Stork, D.G. 2000, *Pattern Classification*, 2nd ed. (New York: Wiley).

Witten, I.H., and Frank, E. 2005, *Data Mining: Practical Machine Learning Tools and Techniques*, 2nd ed. (San Francisco: Morgan Kaufmann).

Mitchell, T.M. 1997, *Machine Learning* (New York: McGraw-Hill).

Vapnik, V. 1998, *Statistical Learning Theory* (New York: Wiley).

Russell, S., and Norvig, P. 2002, *Artificial Intelligence: A Modern Approach*, 2nd ed. (Upper Saddle River, NJ: Prentice-Hall).

Haykin, S. 1998, *Neural Networks: A Comprehensive Foundation*, 2nd ed. (Upper Saddle River, NJ: Prentice-Hall).

Bishop, C.M. 1996, *Neural Networks for Pattern Recognition* (New York: Oxford University Press).

Korb, K.B., and Nicholson, A.E. 2004, *Bayesian Artificial Intelligence* (Boca Raton, FL: Chapman & Hall/CRC).[1]

Neapolitan, R.E. 1990, *Probabilistic Reasoning in Expert Systems* (New York: Wiley).[2]

Jensen, F.V. 2001, *Bayesian Networks and Decision Graphs* (New York: Springer).[3]

16.1 Gaussian Mixture Models and k-Means Clustering

Gaussian mixture models, so called, are one of the simplest examples of classification by *unsupervised learning*. They are also one of the simplest examples where solution by the *EM (expectation-maximization) algorithm* proves highly successful.

Here is the setup: You are given N data points in an M-dimensional space, usually with M in the range one to a few (say, three or four dimensions, tops). You want to "fit" the data, in this special sense: Find a set of K multivariate Gaussian distributions that best represents the observed distribution of data points. The number K is fixed in advance but the means and covariances of the distributions are unknown,

What makes the exercise "unsupervised" is that you are *not told* which of the N data points come from which of the K Gaussians. Indeed, one of the desired *outputs* is, for each data point n, an estimate of the probability that it came from distribution number k. This probability is denoted $P(k|n)$ or p_{nk}, where (using a zero-based counting scheme) $0 \le k < K$ and $0 \le n < N$. The matrix p_{nk} is sometimes called the *responsibility matrix*, because its entries indicate how much "responsibility" component k has for data point n.

Thus, given the data points, say as an $N \times M$ matrix whose rows are vectors of length M, there are a whole bunch of parameters that we want to estimate:

$$\begin{aligned} \boldsymbol{\mu}_k &\quad \text{(the } K \text{ means, each a vector of length } M\text{)} \\ \boldsymbol{\Sigma}_k &\quad \text{(the } K \text{ covariance matrices, each of size } M \times M\text{)} \\ P(k|n) \equiv p_{nk} &\quad \text{(the } K \text{ probabilities for each of } N \text{ data points)} \end{aligned} \tag{16.1.1}$$

We will also get some additional estimates as by-products: $P(k)$ denotes the fraction of all data points in component k, that is, the probability that a data point chosen at random is in k; $P(\mathbf{x})$ denotes the probability (actually a probability density) of finding a data point at some position $\mathbf{x}$, where $\mathbf{x}$ is the M-dimensional position vector; and $\mathcal{L}$ denotes the overall likelihood of the estimated parameter set.

In fact, $\mathcal{L}$ is the key to the whole problem. $\mathcal{L}$ is defined, as usual, as proportional to the probability of the data set, *given* all the fitted parameters. We find the best values for the parameters by maximizing the likelihood $\mathcal{L}$. You can also think of this as maximizing the posterior probability of the parameters, given uniform or very broad priors.

Let's work backward from $\mathcal{L}$. Since the data points are (assumed) independent, $\mathcal{L}$ is the product of the probabilities of finding a point at each observed position $\mathbf{x}_n$,

$$\mathcal{L} = \prod_n P(\mathbf{x}_n) \tag{16.1.2}$$

We can split $P(\mathbf{x}_n)$ into its contribution from each of the K Gaussians and write

$$P(\mathbf{x}_n) = \sum_k N(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) P(k) \tag{16.1.3}$$

where $N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ is the multivariate Gaussian density,

$$N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{M/2} \det(\boldsymbol{\Sigma})^{1/2}} \exp[-\tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu}) \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu})] \tag{16.1.4}$$

$P(\mathbf{x}_n)$ is sometimes called the *mixture weight* of the data point $\mathbf{x}_n$. We can "take apart" $P(\mathbf{x}_n)$ into its K individual contributions, giving the individual probabilities

$$p_{nk} \equiv P(k|n) = \frac{N(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) P(k)}{P(\mathbf{x}_n)} \tag{16.1.5}$$

Equations (16.1.2) through (16.1.5) are a prescription for calculating $\mathcal{L}$ and the p_{nk}'s, given the data, and given values for the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$. In the language of the EM algorithm, this is called an *expectation step* or *E-step*.

But how do we get the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$?

Suppose we *knew* the p_{nk}'s. A familiar theorem for the one-dimensional Gaussian distribution is that the maximum likelihood estimate of its mean is just the arithmetic mean of a set of points drawn from it. This theorem straightforwardly generalizes to yield maximum likelihood estimates for the means, and covariance matrices, of multivariate Gaussians. A further small generalization is that, since we know only probabilistically whether a particular point is drawn from a particular Gaussian, we should count only the appropriate fraction p_{nk} of each point. These considerations result in the following maximum likelihood estimates:

$$\begin{aligned} \hat{\boldsymbol{\mu}}_k &= \sum_n p_{nk} \mathbf{x}_n \Big/ \sum_n p_{nk} \\ \hat{\boldsymbol{\Sigma}}_k &= \sum_n p_{nk} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k) \otimes (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k) \Big/ \sum_n p_{nk} \end{aligned} \tag{16.1.6}$$

and, in a similar vein,

$$\hat{P}(k) = \frac{1}{N} \sum_n p_{nk} \tag{16.1.7}$$

"Hats" here denote estimators; however, this is a notational nicety that we will henceforth ignore. Equations (16.1.6) and (16.1.7) are the so-called *maximization step* or *M-step* of the EM algorithm.

What we have motivated thus far is that *right at* the maximum likelihood solution, both the E-step and the M-step relations will hold. That is, the maximum likelihood parameters are a stationary point for both E-steps and M-steps. The power of the EM algorithm derives from the more powerful theorem (beyond our scope to prove here) that, starting from *any* parameter values, an iteration of E-step followed by an M-step will increase the likelihood value $\mathcal{L}$; and that repeated iterations will converge to (at least a local) likelihood maximum. Often, happily, the convergence is to the global maximum.

The EM algorithm, in brief, is thus

- Guess starting values for the $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and fractions $P(k)$.
- Repeat: An E-step to get new p_{nk}'s and new $\mathcal{L}$, followed by an M-step to get new $\boldsymbol{\mu}_k$'s, $\boldsymbol{\Sigma}_k$'s, and $P(k)$.
- Quit when the value of $\mathcal{L}$ is no longer changing.

One important practical detail is that the values of the Gaussian density function will often be so small as to underflow to zero. It is therefore important to work with logarithms of these densities, rather than the densities themselves, e.g.,

$$\log N(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})\cdot\boldsymbol{\Sigma}^{-1}\cdot(\mathbf{x}-\boldsymbol{\mu}) - \frac{M}{2}\log(2\pi) - \frac{1}{2}\log\det(\boldsymbol{\Sigma}) \quad (16.1.8)$$

A problem arises with equation (16.1.3), where we need to take the sum of quantities, all of which may be so small as to underflow if ever reconstructed from their logarithms. The solution to this problem is the so-called *log-sum-exp* formula,

$$\log\left(\sum_i \exp(z_i)\right) = z_{\max} + \log\left(\sum_i \exp(z_i - z_{\max})\right) \quad (16.1.9)$$

where the z_i's are the logarithms that we are using to represent small quantities and $z_{\max}$ is their maximum. Equation (16.1.9) guarantees that at least one exponentiation won't underflow, and that any that do could have been neglected anyway.

Figure 16.1.1 shows an example of how the EM algorithm converges to a solution with 1000 two-dimensional data points and four components. As the number of data points increases, the topography of the likelihood space gets smoother, with fewer local minima, so that it becomes more and more likely that the global maximum will be found (as in this case).

You should always inspect an EM solution for reasonableness. If you are getting hung up on an unacceptable local maximum, one strategy is to do a series of independent runs, using K randomly chosen data points as the starting means in each case. (Be sure that you don't duplicate a data point in the starting guesses.) Then pick the best one, i.e., the one that converges to the largest log-likelihood.

Here is a structure that implements the EM algorithm for Gaussian mixture models, given only the data points and initial estimates of the means $\boldsymbol{\mu}_k$. The constructor sets the problem up, and does one initial E-step and M-step. Thereafter, the user alternately calls the `estep()` and `mstep()` routines, until convergence is achieved, as signaled by the return value of `estep()`, the change in log-likelihood, becoming sufficiently small (say, 10^{-6}). The results are then available in the structure members `means`, `resp`, `frac`, and `sig`.

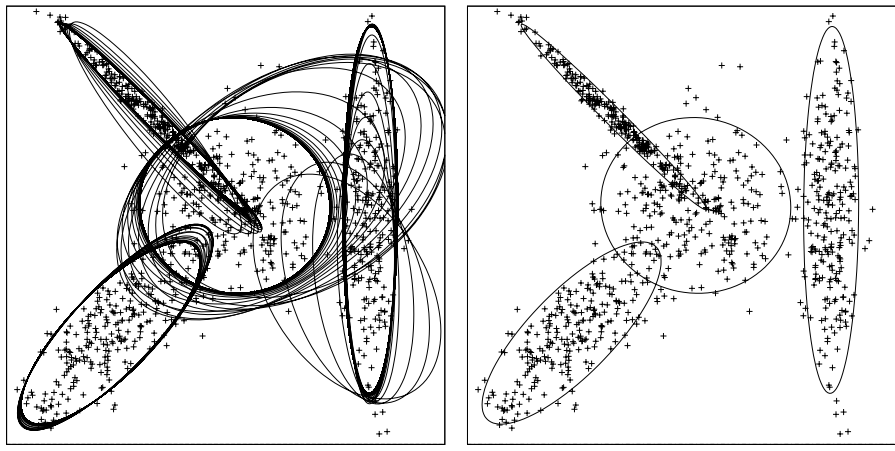


Figure 16.1.1. Example of a Gaussian mixture model in $M = 2$ dimensions, with $N = 1000$ data points and $K = 4$ components. Left: Evolution of the estimated means and covariances, shown as 2-sigma ellipses. The ellipses are plotted after each iteration of an E-step and M-step (see text). Right: The converged result. The leftmost two components converged rapidly. The rightmost component took about 10 iterations to get to near-convergence; only after it had done so did the central component shrink down to a converged result.

gaumixmod.h

```
struct preGaumixmod {
```
For nonwizards, this is basically a typedev of `Mat_mm` as an `mm`×`mm` matrix. For wizards, what is going on is that we need to set a static variable `mmstat` *before* defining `Mat_mm`, and this must happen *before* the `Gaumixmod` constructor is invoked.
```
    static Int mmstat;
    struct Mat_mm : MatDoub {Mat_mm() : MatDoub(mmstat,mmstat) {} };
    preGaumixmod(Int mm) {mmstat = mm;}
};
Int preGaumixmod::mmstat = -1;

struct Gaumixmod : preGaumixmod {
```
Solve for a Gaussian mixture model from a set of data points and initial guesses of k means.
```
    Int nn, kk, mm;                 Nos. of data points, components, and dimensions.
    MatDoub data, means, resp;      Local copies of x_n's, μ_k's, and the p_nk's.
    VecDoub frac, lndets;           P(k)'s and log det Σ_k's.
    vector<Mat_mm> sig;             Σ_k's
    Doub loglike;                   log ℒ.
    Gaumixmod(MatDoub &ddata, MatDoub &mmeans) : preGaumixmod(ddata.ncols()),
    nn(ddata.nrows()), kk(mmeans.nrows()), mm(mmstat), data(ddata), means(mmeans),
    resp(nn,kk), frac(kk), lndets(kk), sig(kk) {
```
Constructor. Arguments are the data points (as rows in a matrix) and initial guesses for the means (also as rows in a matrix).
```
        Int i,j,k;
        for (k=0;k<kk;k++) {
            frac[k] = 1./kk;            Uniform prior on P(k).
            for (i=0;i<mm;i++) {
                for (j=0;j<mm;j++) sig[k][i][j] = 0.;
                sig[k][i][i] = 1.0e-10;     See text at end of this section.
            }
        }
        estep();                        Perform one initial E-step and M-step. User
        mstep();                            is responsible for calling additional steps
    }                                       until convergence is obtained.
    Doub estep() {
```
Perform one E-step of the EM algorithm.
```
        Int k,m,n;
```

```
        Doub tmp,sum,max,oldloglike;
        VecDoub u(mm),v(mm);
        oldloglike = loglike;
        for (k=0;k<kk;k++) {                Outer loop for computing the p_nk's.
            Cholesky choltmp(sig[k]);       Decompose Σ_k in the outer loop.
            lndets[k] = choltmp.logdet();
            for (n=0;n<nn;n++) {            Inner loop for p_nk's.
                for (m=0;m<mm;m++) u[m] = data[n][m]-means[k][m];
                choltmp.elsolve(u,v);       Solve L · v = u.
                for (sum=0.,m=0; m<mm; m++) sum += SQR(v[m]);
                resp[n][k] = -0.5*(sum + lndets[k]) + log(frac[k]);
            }
        }
        At this point we have unnormalized logs of the p_nk's. We need to normalize using
        log-sum-exp and compute the log-likelihood.
        loglike = 0;
        for (n=0;n<nn;n++) {                    Separate normalization for each n.
            max = -99.9e99;                     Log-sum-exp trick begins here.
            for (k=0;k<kk;k++) if (resp[n][k] > max) max = resp[n][k];
            for (sum=0.,k=0; k<kk; k++) sum += exp(resp[n][k]-max);
            tmp = max + log(sum);
            for (k=0;k<kk;k++) resp[n][k] = exp(resp[n][k] - tmp);
            loglike +=tmp;
        }
        return loglike - oldloglike;            When abs of this is small, then we have
    }                                               converged.
    void mstep() {
    Perform one M-step of the EM algorithm.
        Int j,n,k,m;
        Doub wgt,sum;
        for (k=0;k<kk;k++) {
            wgt=0.;
            for (n=0;n<nn;n++) wgt += resp[n][k];
            frac[k] = wgt/nn;                   Equation (16.1.7).
            for (m=0;m<mm;m++) {
                for (sum=0.,n=0; n<nn; n++) sum += resp[n][k]*data[n][m];
                means[k][m] = sum/wgt;          Equation (16.1.6).
                for (j=0;j<mm;j++) {
                    for (sum=0.,n=0; n<nn; n++) {
                        sum += resp[n][k]*
                            (data[n][m]-means[k][m])*(data[n][j]-means[k][j]);
                    }
                    sig[k][m][j] = sum/wgt;     Equation (16.1.6).
                }
            }
        }
    }
};
```

About the only place that `Gaumixmod` can fail algorithmically (as distinct from converging to a poor, local, solution) is by encountering a zero or negative diagonal element in the Cholesky decomposition. As a result, all sins tend to appear, sometimes confusingly, as exceptions at that point in the code. If you are getting such exceptions, here are some possibilities:

- You have duplicated vectors in your initial guesses for the $\boldsymbol{\mu}_k$'s.
- One or more of your $\boldsymbol{\mu}_k$'s is so distant from all data points that it is not "attracting" enough of them to solve for the parameters of its component. Try using random data points as starting guesses, or reduce K.
- You may just have too few data points N to support a nondegenerate model with K components. Reduce K or get more data!

- Rarely, you might want to change the constant `1.0e-10` that initializes the diagonal components of $\mathbf{\Sigma}_k$ in the code. (See discussion under "K-Means Clustering," below.)
- You can reduce the number of parameters in $\mathbf{\Sigma}$, as we now discuss.

Occasionally data are too sparse, or too noisy, to give meaningful results for all the components of the covariance matrices $\mathbf{\Sigma}_k$. In such cases, you can impose simpler covariance models by changing the re-estimation formulas for $\mathbf{\Sigma}$ in equation (16.1.6). One step of simplification is to make $\mathbf{\Sigma}$ diagonal, while still allowing different variances for the different dimensions. The re-estimation formula for the diagonal components of $\mathbf{\Sigma}_k$ is then

$$(\hat{\mathbf{\Sigma}}_k)_{mm} = \sum_n p_{nk}[(\mathbf{x}_n)_m - (\hat{\boldsymbol{\mu}}_k)_m]^2 \Big/ \sum_n p_{nk} \tag{16.1.10}$$

where subscripts m indicate that particular component of the vector. Set nondiagonal components of $\mathbf{\Sigma}_k$ to zero.

Even more drastic, we can replace $\mathbf{\Sigma}_k$ by a single scalar (that is, spherical) variance by using the re-estimation formula

$$(\hat{\mathbf{\Sigma}}_k) = \mathbf{1} \times \left(\sum_n p_{nk} |\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k|^2 \Big/ \sum_n p_{nk} \right) \tag{16.1.11}$$

where $\mathbf{1}$ is the identity matrix.

We have not coded these options in `Gaumixmod`, but they are easy to add.

16.1.1 A Note on the Use of Cholesky Decomposition

It is worth remarking briefly on the use of Cholesky decomposition (§2.9) in this and similar manipulations of multivariate Gaussians.

In the `Gaumixmod` routine above, we need a way of inverting the covariance matrices — or, more precisely, an efficient way to compute expressions like $\mathbf{y} \cdot \mathbf{\Sigma}^{-1} \cdot \mathbf{y}$. Because the covariance matrix $\mathbf{\Sigma}$ is symmetric and positive-definite, the Cholesky decomposition, which has fewer operations than other methods, can be used, giving

$$\mathbf{\Sigma} = \mathbf{L} \cdot \mathbf{L}^T \tag{16.1.12}$$

where $\mathbf{L}$ is a lower triangular matrix, implying

$$Q = \mathbf{y} \cdot \mathbf{\Sigma}^{-1} \cdot \mathbf{y} = \left| \mathbf{L}^{-1} \cdot \mathbf{y} \right|^2 \tag{16.1.13}$$

Since $\mathbf{L}$ is triangular, $\mathbf{L}^{-1} \cdot \mathbf{y}$ can be obtained efficiently by backsubstitution.

Another very convenient use for the decomposition (16.1.12) is in the mundane task of drawing error ellipses, as in Figure 16.1.1 (or, similarly, error ellipsoids in three dimensions). The locus of points $\mathbf{x}$ that are one standard deviation ("1-sigma") away from the mean $\boldsymbol{\mu}$ is given by

$$1 = (\mathbf{x} - \boldsymbol{\mu}) \cdot \mathbf{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \quad \Rightarrow \quad \left| \mathbf{L}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \right| = 1 \tag{16.1.14}$$

Now suppose that $\mathbf{z}$ is a point on the unit circle (two dimensions) or unit sphere (three dimensions). Then, by substitution into equation (16.1.14), you can easily see that

$$\mathbf{x} = \mathbf{L} \cdot \mathbf{z} + \boldsymbol{\mu} \tag{16.1.15}$$

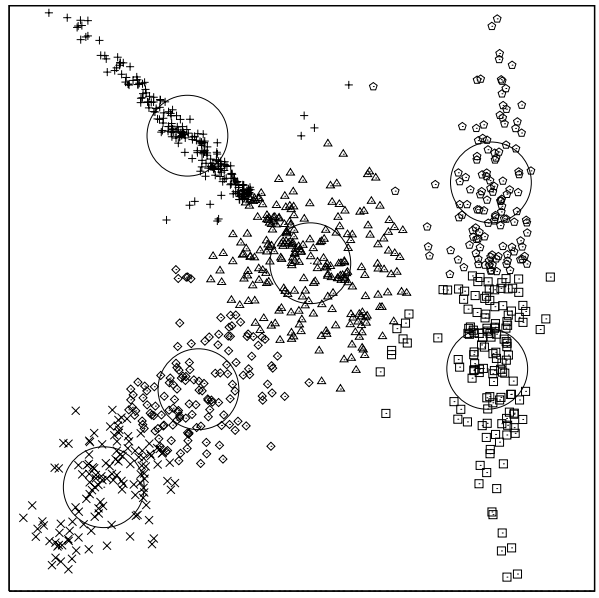


Figure 16.1.2. Output of k-means clustering as applied to the same data as Figure 16.1.1, with $K = 6$ components. The final assignments are shown as different plotted symbols. The centers of the large circles are the locations of the final means. (The radius of those circles is arbitrary, for visibility only.) Unlike Gaussian mixture modeling, k-means clustering can't split a point probabilistically between two components, so many points from the Gaussian at the upper left are mistakenly assigned to the central component. Also, k-means clustering needs more than one component to model a Gaussian with a large aspect ratio, because it clusters by radial distance.

is a point on the 1-sigma locus. Going around the unit circle in $\mathbf{z}$, and using the mapping (16.1.15), gives the desired ellipse. Put a constant 2 in front of the $\mathbf{L}$ in (16.1.15) for 2-sigma ellipses, and so forth.

We already remarked, in §7.4, on the closely related use of Cholesky decomposition to generate multivariate Gaussian deviates from a given covariance matrix.

16.1.2 K-Means Clustering

One interesting simplification of Gaussian mixture modeling has an independent history and is known as *k-means clustering*. We forget about the $\mathbf{\Sigma}_k$ covariances matrices completely, and we forget about probabilistic assignments of data points to components. Instead, each data point gets assigned to one (and only one) of the K components.

The E-step is simply: Assign each data point $\mathbf{x}_n$ to the component k whose mean $\boldsymbol{\mu}_k$ it is closest to, by Euclidean distance.

The M-step is simply: For all k, re-estimate the mean $\boldsymbol{\mu}_k$ as the average of data points $\mathbf{x}_n$ assigned to component k.

The convergence criterion is: Stop when an E-step doesn't change the assignment of any data point (in which case the M-step would also produce unchanged $\boldsymbol{\mu}_k$'s).

Interestingly, convergence is guaranteed — you can't get into an infinite loop of, say, shifting a point back and forth between two components. Despite its simplicity, k-means clustering can be quite useful: It is very fast, and it converges very

rapidly. It can be used as a method to reduce a large number of data points to a much smaller number of "centers," which can then be used as starting points for more sophisticated methods.

For example, you might use k-means clustering to get starting values for a Gaussian mixture model that has difficulty converging to a good, global maximum. If K is the ultimate number of components that you want, you might use k-means to get down to $\sim 3 \times K$ components, then (repeatedly) randomly select K of these as starting guesses for the Gaussian model.

Be alert to the fact that k-means clustering has an intrinsically "spherical" view of the world, because of its Euclidean "nearest-to" assignments. If you have components that might have big aspect ratios, be sure to set K large enough so that these can be represented by several different centers. Figure 16.1.2 shows the same input data as Figure 16.1.1, now clustered by k-means. The Gaussians at the lower left and right have broken up into two centers each. The Gaussian at the upper left is only a single component, because it has had many of its points misclassified into the central component. (A Gaussian mixture model would have assigned those points probabilistically to both components.)

Code for k-means classification is similar to, but much shorter than, the previous code for a Gaussian mixture model:

kmeans.h

```
struct Kmeans {
```
Solve for a k-means clustering model from a set of data points and initial guesses of the means. Output is a set of means and an assignment of each data point to one component.
```
    Int nn, mm, kk, nchg;
    MatDoub data, means;
    VecInt assign, count;
    Kmeans(MatDoub &ddata, MatDoub &mmeans) : nn(ddata.nrows()), mm(ddata.ncols()),
    kk(mmeans.nrows()), data(ddata), means(mmeans), assign(nn), count(kk) {
```
Constructor. Arguments are the data points (as rows in a matrix), and initial guesses for the means (also as rows in a matrix).
```
        estep();
        mstep();
    }
```
Perform one initial E-step and M-step. User is responsible for calling additional steps until convergence is obtained.
```
    Int estep() {
```
Perform one E-step.
```
        Int k,m,n,kmin;
        Doub dmin,d;
        nchg = 0;
        for (k=0;k<kk;k++) count[k] = 0;
        for (n=0;n<nn;n++) {
            dmin = 9.99e99;
            for (k=0;k<kk;k++) {
                for (d=0.,m=0; m<mm; m++) d += SQR(data[n][m]-means[k][m]);
                if (d < dmin) {dmin = d; kmin = k;}
            }
            if (kmin != assign[n]) nchg++;
            assign[n] = kmin;
            count[kmin]++;
        }
        return nchg;
    }
    void mstep() {
```
Perform one M-step.
```
        Int n,k,m;
        for (k=0;k<kk;k++) for (m=0;m<mm;m++) means[k][m] = 0.;
        for (n=0;n<nn;n++) for (m=0;m<mm;m++) means[assign[n]][m] += data[n][m];
        for (k=0;k<kk;k++) {
```

```
            if (count[k] > 0) for (m=0;m<mm;m++) means[k][m] /= count[k];
        }
    }
};
```

Incidentally, k-means clustering is not only a simplification of Gaussian mixture models; it is actually a limiting case. If the $\mathbf{\Sigma}_k$ matrices are all held fixed as

$$\mathbf{\Sigma}_k = \epsilon\, \mathbf{1} \tag{16.1.16}$$

with ϵ infinitesimal and $\mathbf{1}$ the identity matrix, then the component k with mean closest to $\mathbf{x}_n$ will be assigned all of the responsibility p_{nk} for that n. The re-estimation of the $\boldsymbol{\mu}_k$'s then is identical to k-means clustering. The theorem that proves that the EM algorithm converges for Gaussian mixtures can easily be modified to prove the convergence of k-means clustering. (Basically, there is a hidden log-likelihood function that can be shown to increase at each step.)

Indeed, we can now explain the obscure constant `1.0e-10` in the initialization part of `Gaumixmod`: It is a value for ϵ that makes that routine's *first* E-step, M-step iteration be one of k-means clustering.

CITED REFERENCES AND FURTHER READING:

McLachlan, G. and Peel, D. 2000, *Finite Mixture Models* (New York: Wiley).

Moore, A.W. 2004, "Clustering with Gaussian Mixtures," at `http://www.cs.cmu.edu/~awm`.

Dempster, A.P., Laird, N.M., and Rubin, D.B. 1977, "Maximum Likelihood from Incomplete Data via the EM Algorithm," *Journal of the Royal Statistical Society*, Series B, vol. 39, pp. 1-38. [The original paper on EM methods.]

Tanner, M.A. 2005, *Tools for Statistical Inference: Methods for the Exploration of Posterior Distributions and Likelihood Functions*, 3rd ed. (New York: Springer).

16.2 Viterbi Decoding

In this section we discuss models with discrete states, and how to use data to estimate what state a model is in, or what succession of states it traverses by allowed transitions. By *state*, we mean some discrete condition that can be characterized as a node on a directed graph like that in Figure 16.2.1. By *transition*, we mean moving along one of the directed edges of the graph. If you want to characterize a continuous variable in the context of this section, you need to define a set of discrete bins for its possible values, and make these the states.

The setup we describe is slightly more general than its close cousin, the directed graph of *stages and states* that defined the dynamic programming (DP) problem in §10.13. For some applications, the estimation problem of interest does live on a graph that has states and stages, exactly like DP; but for other applications, we need a general directed graph. We'll consider both types below.

Historically, problems involving the estimation of states have arisen in diverse, and often noncommunicating, fields. There are often multiple names for single concepts. (We saw this previously in the Bellman-Dijkstra-Viterbi algorithm for DP.) This history also makes it hard to give, in this section, a unified treatment with a single narrative. A more practical approach is to go through a couple of examples from different fields, and then, afterward, make some comparisons and give some advice.

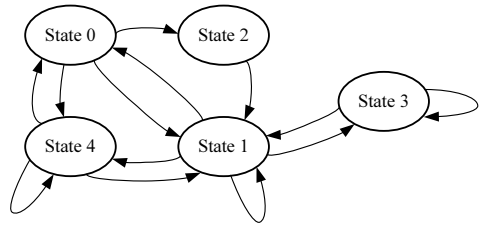


Figure 16.2.1. Graph of a dynamical system with five states. Allowed transitions are shown by arrows.

16.2.1 Error-Correcting Codes and Soft-Decision Decoding

An (N, K) *binary block code* is a list of 2^K binary *codewords*, each of length $N > K$ bits, designed to send a K-bit message in such a way that it can be received correctly even if one or more of the N bits arrive garbled (that is, 0 instead of 1, or vice versa). Two simple examples are shown below. In these particular cases, the message bits are the initial bits of the codeword, but that need not be true in general. Assigning any permutation of the codewords to message words is, effectively, the same code; likewise an arbitrary permutation of the bits in all the codewords (permuting bit-columns in the table).

(6,3) Shortened Hamming	
message	codeword
000	000000
001	001110
010	010101
011	011011
100	100011
101	101101
110	110110
111	111000

(7,4) Hamming	
message	codeword
0000	0000000
0001	0001011
0010	0010111
0011	0011100
0100	0100110
0101	0101101
0110	0110001
0111	0111010
1000	1000101
1001	1001110
1010	1010010
1011	1011001
1100	1100011
1101	1101000
1110	1110100
1111	1111111

Both of the codes shown have the property that their *Hamming distance* is 3. This means that all pairs of codewords differ in at least three bits. This is the property of the code that makes it "error correcting on one bit." If you receive a codeword with one of its bits wrong, then (i) it will not be in the above table, and (ii) there will be a unique codeword in the table that differs from it in one bit position. So, trying each bit position in turn, you can figure out what was the intended codeword.

A longer code, with a larger Hamming distance d, can be error correcting for more than one garbled bit, in fact for $(d-1)/2$ bits (rounding down). An (N, K) code can have d as large as $N - K$. However, trying all possible corrections until you find a valid codeword is a very poor decode strategy!

For so-called *linear codes* it is possible to construct a *parity-check matrix* $\mathbf{P}$ with the property that multiplying it by the vector of received bits (and doing all

arithmetic modulo 2) gives a vector, the so-called *syndrome*, that is either all zeros (indicating that the received bits are ok) or else it uniquely corresponds to a mask (termed a *coset leader*) that tells which bits need correcting. So this error-correction algorithm, called *syndrome decoding*, can be summarized as:

- multiply the received bits by the parity-check matrix to get the syndrome,
- do a table lookup of the syndrome to get the coset leader, and
- XOR the coset leader with the received bits to get a valid codeword.

For example, the parity check matrix for the $(7, 4)$ Hamming code, above, is

$$\mathbf{P} = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \tag{16.2.1}$$

(Convert codewords to column vectors by reading from left to right.) The lookup table relating syndrome to coset leader is

syndrome	coset leader
000	0000000
001	0000001
010	0000010
011	0001000
100	0000100
101	1000000
110	0100000
111	0010000

This particular code is called a *perfect code* because the number of syndromes exactly equals the number of coset leaders with one nonzero bit, plus 1 for the zero syndrome, a numerological coincidence. There are very few perfect codes, because there are very few sets of integers satisfying

$$1 + \binom{N}{1} + \cdots + \binom{N}{e} = 2^{N-K} \tag{16.2.2}$$

where e is the number of bits corrected. Probably the most nontrivial perfect code is the *Golay code*, with $N = 23$, $K = 12$, and $e = 3$. (Check out the numerology yourself.)

It's no big deal if a code is not perfect. It just means that there are some extra syndromes that correct *some* errors of more than e bits, but not enough to correct all such errors. You include these extra syndromes in the table, and run the algorithm exactly as already described. However, if a code is too far from perfect, you are wasting syndromes without gaining more bits of sure correction.

In practical applications, N and K are larger than these examples. For example, the lowest level of error correction on an audio compact disk (CD) is a $(28, 24)$ *Reed-Solomon* (RS) code, which can correct $e = 2$ bits. (On a CD, bits of the output codewords from many consecutive blocks are then interleaved and further protected by an RS(32,28) code.) Reed-Solomon codes are typically decoded by a more efficient process than syndrome decoding, using the so-called *Berlekamp-Massey* algorithm.

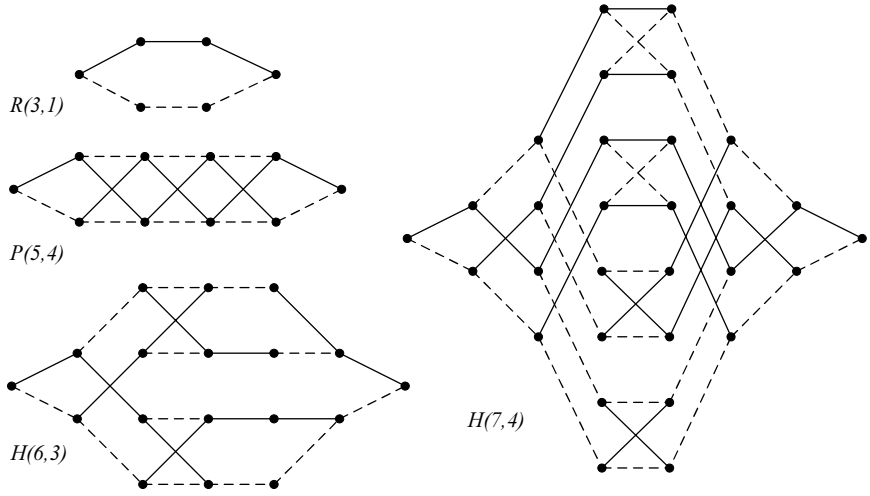


Figure 16.2.2. Trellises associated with four binary codes. The graph is traversed from left to right. A zero is output when any dotted edge is traversed, a one for a solid edge. Every path yields a valid codeword.

Now take a deep breath. Everything we have discussed thus far is what is called *hard-decision decoding* (HDD), meaning that hard decisions are made as to whether each incoming bit is a 1 or 0, with the error-correction algorithm acting on the resulting, possibly garbled, codeword. Virtually all coding theory utilized HDD until the early 1970s. Then came the giant leap forward with the recognition by multiple practitioners that Viterbi's 1967 decoding algorithm (an independent rediscovery of Bellman-Dijkstra, we might now say) could utilize "soft" data about each bit as easily as hard.

To understand *soft-decision decoding* (SDD), let us first note that every binary code can be represented by the kind of stage/state graph that we met in dynamic programming (§10.13), which, in the present context, is called a *trellis*. Figure 16.2.2 shows the trellises for the two codes given explicitly above, as well as for the schoolbook examples of a repetition code ("tell me three times") and a parity code. The latter, with $d = 1$, is error detecting, but not error correcting.

Although arrows are not shown, the trellis is traversed from left to right. Any such path on the trellis generates a valid codeword. A zero bit is emitted when a dotted edge is traversed, a one bit for a solid edge. You encode message bits by deciding whether to branch up or down, when you have a choice. Notice that you don't get such a choice at every stage: "Forced" edges generate precisely the extra codeword bits that the code's redundancy requires.

Although every code has a trellis, it is not so easy to find the *minimal trellis*, the one that has the fewest possible states at its maximum expansion. MacKay [1] gives a brief introduction; many additional references are in [3].

The first great idea behind soft-decision decoding is that we don't need to decide whether an incoming bit is a 0 or 1. Rather, we just need to assign a probability to each possibility (summing to unity, of course). For example, a bit's value may be determined by whether an instantaneous voltage is positive or negative — but the voltage measurement has some Gaussian spread of errors. If the voltage is many standard deviations positive, or negative, then the respective probabilities are very

close to one or zero; but if the voltage is only $t = 0.5$ (say) standard deviations away from zero, we may want to assign a probability of 0.6915 to one more favored outcome, and 0.3085 to the less favored, since

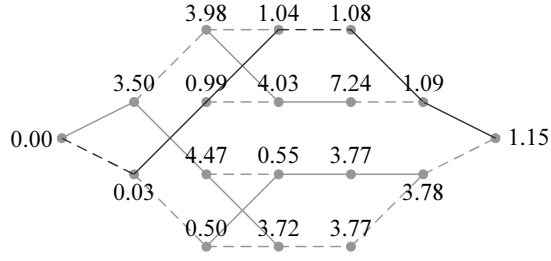
$$\frac{1}{\sqrt{2\pi}} \int_{-\infty}^{0.5} e^{-z^2/2} dz \approx 0.6915 \tag{16.2.3}$$

(By the end of this section we will be more sophisticated about the notion of assigning probabilities to transitions.)

The second great idea is that the problem of finding the maximum likelihood path through a trellis — that is, the path with the maximum product of the probabilities at each stage — is just a dynamic programming problem, where the cost of traversing an edge whose probability is p is taken as $-\log(p)$, a positive number, since $0 \le p \le 1$. The minimum cost path, with this metric, is the maximum likelihood path. In each stage, all the 0 edges (dotted lines in the figure) get one probability, and all the 1 edges (solid lines) get its complement. The edge probabilities can, and in general will, vary with each bit received (that is, from stage to stage), since the noise and path loss can vary with time.

Take these two ideas together and you have *soft-decision decoding using the Viterbi algorithm*.

The following tableau decodes one codeword for the shortened Hamming (6, 3) code given above. In this example, five of the six bits are received fairly unambiguously, while one bit (the second) is seen to be rather ambiguous. Nevertheless, the algorithm treats all bits equally "softly." Reviewing §10.13 as necessary, you should be able to see where all the numbers in the tableau come from, as well as how the darker path (which is the final "hard" decision for the codeword 011011) is obtained by backtracking. Given the appropriate cost function, the routine `dynpro` in §10.13 does exactly this calculation.



Prob(0) =	.97	.62	.04	.96	.01	.06
Prob(1) =	.03	.38	.96	.04	.99	.94
–log[Prob(0)] =	0.03	0.47	3.22	0.04	4.61	2.81
–log[Prob(1)] =	3.50	0.96	0.05	3.21	0.01	0.06

You might have the "bright idea" of converting the final minimum path length, 1.15 in the above example, into a probability by taking its negative exponential. The result is 0.3166. Does this mean that you'll get the right codeword only 31% of the time? No! Go stand in the corner! You are confusing likelihood with probability. The likelihoods of all eight codewords (not found by the DP algorithm, but computed exhaustively) are, for this example,

codeword	likelihood
000000	0.000014
001110	0.001372
010101	0.000006
011011	0.316126
100011	0.000665
101101	0.000007
110110	0.000001
111000	0.000006

You can see that the likelihoods don't add up to 1, and that the favored path wins over any competitor by a large factor. (Bayesians: We know that this paragraph is making you break out in a rash. We are on your side, and will have more to say about this in §16.3.4.)

In the above example, the second bit was merely ambiguous. What if it had instead been really wrong, indicating, say, 0.99 probability of having a value zero? No problem. Since the underlying code is one-bit error correcting, the DP algorithm will readily decide to traverse the unlikely single edge, since the alternative would be to traverse two or more unlikely edges on other bits. However, if we made the second bit incorrect with probability 0.999999, the algorithm would "correct" two other bits instead, which, under the circumstances, would be the best decision.

You can see that it is not meaningful to say exactly how many bits e a soft-decision decode algorithm can correct. It just makes the best choice determined by the probabilities. As another example, we might consider the simple parity code shown in Figure 16.2.2. With a hard-decision decode, parity does not give enough information to correct a single bit. With a soft-decision algorithm, however, the parity bit can cast the deciding vote if some other bit is wavering too close to an ambiguous 50% probability level.

Soft-decision decoding algorithms are available for essentially all codes in use today, including Reed-Solomon codes and the important *turbo codes* [2] that are beyond our scope. Some important applications (e.g., *trellis coded modulation*), use short trellises whose end states loop around to become identified with their start states. The Viterbi algorithm is applied to long sequences of input symbols that loop through the trellis many times. In trellis coded modulation, the symbols being (softly) decoded are not single bits, but locations in the complex phase plane that comprise a carefully chosen *constellation* centered at the origin (for example, a hexagonal lattice).

CITED REFERENCES AND FURTHER READING:

Lin, S. and Costello, D.J. 2004, *Error Control Coding*, 2nd ed. (Upper Saddle River, NJ: Pearson-Prentice Hall).

Blahut, R.E. 2002, *Algebraic Codes for Data Transmission* (Cambridge, UK: Cambridge University Press).

MacKay, D.J.C. 2003, *Information Theory, Inference, and Learning Algorithms* (Cambridge, UK: Cambridge University Press).[1]

Schlegel, C. and Perez, L. 2000, *Trellis and Turbo Coding*, (Piscataway, NJ: IEEE Press).[2]

"Special Issue on Codes and Complexity", 1996, *IEEE Transactions on Information Theory*, vol. 42, no. 6, pp. 1649-2064.[3]

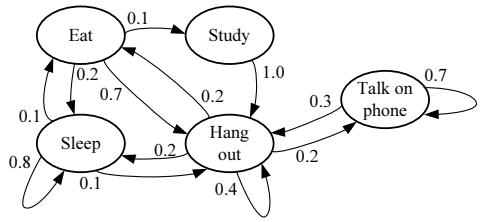


Figure 16.3.1. Example of a Markov model. Transitions occur between states along the directed edges shown. Each outgoing edge is labeled by its probability. The sum of the probabilities on the outgoing edges from each state is 1. This model is called "Teen Life."

16.3 Markov Models and Hidden Markov Modeling

Trellises, like those in §16.2, are directed graphs without any loops, so a path that begins at the leftmost node inevitably ends, after a finite number of stages, at the rightmost node. The more general *Markov model* lives on a graph that can have loops (as in Figure 16.2.1), so some paths can continue indefinitely. Indeed, as a convention, one can add a self-loop (a directed edge connecting a state to itself) to any state that would otherwise have "no way out." Then, all paths can be continued indefinitely, even those whose fate is to remain stuck in a single state.

To turn such a directed graph into a Markov model (also known as a *Markov chain* or *first-order Markov process*), we simply label all of its edges with *transition probabilities*, such that the sum of probabilities over the outgoing edges from each node is 1. Figure 16.3.1 shows an example, a Markov model with five states, that we call "Teen Life."

A single realization of a Markov model is a random path that moves from state to state according to the model's probabilities. These are conveniently organized into a *transition matrix* $\mathbf{A}$ whose element A_{ij} is the probability associated with an $i \to j$ transition, that is, the probability of moving to state j, given state i as the starting point. A valid transition matrix satisfies

$$0 \le A_{ij} \le 1 \qquad \text{and} \qquad \sum_j A_{ij} = 1 \tag{16.3.1}$$

The transition matrix for Teen Life (Figure 16.3.1) is

$$\mathbf{A} = \begin{pmatrix} 0 & 0.7 & 0.1 & 0 & 0.2 \\ 0.2 & 0.4 & 0 & 0.2 & 0.2 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0.3 & 0 & 0.7 & 0 \\ 0.1 & 0.1 & 0 & 0 & 0.8 \end{pmatrix} \tag{16.3.2}$$

where the states are numbered in the order (Eat, Hang, Study, Talk, and Sleep).

A routine for generating a realization of a Markov model from its $M \times M$ transition matrix, using the `Ran` structure of §7.1 to get random numbers, is straightforward.

markovgen.h

```
void markovgen(const MatDoub_I &atrans, VecInt_O &out, Int istart=0,
    Int seed=1) {
```

Generate a realization of an M-state Markov model, given its $M \times M$ transition matrix `atrans`. The vector `out` is filled with integers in the range $0 \ldots M-1$. The starting state is the optional argument `istart` (defaults to 0). `seed` is an optional argument that sets the seed of the random number generator.

```
    Int i, ilo, ihi, ii, j, m = atrans.nrows(), n = out.size();
    MatDoub cum(atrans);                Temporary matrix to hold cumulative probabilities.
    Doub r;
    Ran ran(seed);                      Use the random number generator Ran.
    if (m != atrans.ncols()) throw("transition matrix must be square");
    for (i=0; i<m; i++) {               Fill cum and die if clearly not a transition matrix.
        for (j=1; j<m; j++) cum[i][j] += cum[i][j-1];
        if (abs(cum[i][m-1]-1.) > 0.01)
            throw("transition matrix rows must sum to 1");
    }
    j = istart;                         The current state is kept in j.
    out[0] = j;
    for (ii=1; ii<n; ii++) {            Main loop.
        r = ran.doub()/cum[j][m-1];     Slightly-off normalization gets corrected here.
        ilo = 0;
        ihi = m;
        while (ihi-ilo > 1) {           Use bisection to find location among the cumu-
            i = (ihi+ilo) >> 1;             lative probabilities.
            if (r>cum[j][i-1]) ilo = i;
            else ihi = i;
        }
        out[ii] = j = ilo;              Set new current state.
    }
}
```

What makes the transition matrix a matrix, and not just a table of probabilities, is its connection to *ensembles* of realizations of the corresponding Markov model. An ensemble can be characterized by the components of a *population vector* $\mathbf{s}_t$ whose components give the number of models in each state at time t. (Here and below, we use t as an integer, discrete time variable. On a trellis it would be called a *stage* instead of a *time*.) For Teen Life, we can give names to the components of $\mathbf{s}_t$ corresponding to the states: (E, H, S, T, Z).

If all the models in the ensemble are evolved by one timestep (one transition), then the population vector $\mathbf{s}_t$ turns into $\mathbf{s}_{t+1}$ by the matrix multiplication

$$\mathbf{s}_{t+1} = \mathbf{A}^T \, \mathbf{s}_t \tag{16.3.3}$$

The transpose operation is needed only because two common conventions are at odds: The time order $i \to j$ for A_{ij} versus the left matrix multiplication of a column vector (whose implicit time ordering is "from" the second index "to" the first). Given a matrix, you can easily tell whether it is intended to be an $\mathbf{A}$ or an $\mathbf{A}^T$, by whether, respectively, its rows or columns sum to unit probability.

Note that we can evolve more than one step at a time, by precomputing powers of $\mathbf{A}^T$. So, $\mathbf{s}_{t+n} = (\mathbf{A}^T)^n \, \mathbf{s}_t$, for example.

Every Markov model has at least one equilibrium distribution of states that remains unaffected when multiplied by $\mathbf{A}^T$. To prove this, we write

$$\mathbf{A}^T \, \mathbf{s}_e = \mathbf{s}_e \qquad \Longleftrightarrow \qquad (\mathbf{A}^T - \mathbf{1}) \, \mathbf{s}_e = \mathbf{0} \tag{16.3.4}$$

where $\mathbf{s}_e$ is the sought-after equilibrium state. Equation (16.3.4) can hold if and only if $\mathbf{A}^T - \mathbf{1}$ is a singular matrix. Since the columns of $\mathbf{A}^T$ all sum to 1, the columns

of $\mathbf{A}^T - \mathbf{1}$ all sum to zero, and hence are linearly dependent, q.e.d. Equivalently, we have proved that $\mathbf{A}^T$ has at least one eigenvalue equal to 1. The corresponding eigenvector is an equilibrium distribution of states. If there is only one eigenvalue equal to 1, the equilibrium is unique. For the Teen Life model, there is one eigenvalue of 1, and the corresponding eigenvector (normalized so that its components sum to unity) is approximately $(0.099, 0.297, 0.001, 0.198, 0.395)$. (This teenager spends about 39.5% of his/her time sleeping, 19.8% talking on the phone, 0.1% studying, and so forth.)

Do almost all starting distributions converge to a unique equilibrium, in which case the model is said to be *ergodic*? Not necessarily. Two things can go wrong. First, if there is more than one eigenvalue equal to 1, a model will converge to some different linear combination of the corresponding eigenvectors for different starting distributions. Such models are said to fail the test of *irreducibility*. Second, the model may have a periodic limit cycle, so that, for most starting distributions, it doesn't converge at all. Such models are said to fail the test of *aperiodicity*. The theorem (and vocabulary test) is: Irreducibility and aperiodicity imply ergodicity.

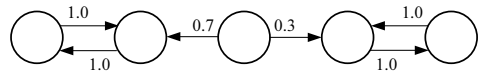
One way to diagnose these conditions is to perform successive squarings of the matrix $\mathbf{A}^T$ to take it to a very high power, say 2^{32}. This requires $O(32\,M^3)$ operations for a model with M states. (While there are more sophisticated methods, none scale better than M^3.) If all M columns in the result are converging to identical vectors, then there is just one eigenvalue (unity), and all starting distributions will converge to its eigenvector (which is in fact the repeated column vector). The model is then ergodic.

Otherwise, locate any rows in the power matrix that are zero, and cross out those rows and their corresponding columns. (These are states that become permanently unpopulated as the model evolves.) Then, check to see if the remaining columns are all eigenvectors with unit eigenvalue. You can do the test by multiplying each such column by the original $\mathbf{A}^T$. If all columns pass the test, then there are multiple equilibria, but all starting distributions will converge to some combination of them. If any column fails the test, then the model has a periodic limit cycle. There are still equilibria, given by the eigenvectors of unit eigenvalue, but a starting state must be very special to evolve to one. Indeed, such states are a set of measure zero, and we can say that the equilibria are *unstable*.

A simple example with multiple equilibria and periodic limit cycles is the transition matrix

$$\mathbf{A}' = \begin{pmatrix} 0 & 1.0 & 0 & 0 & 0 \\ 1.0 & 0 & 0 & 0 & 0 \\ 0 & 0.7 & 0 & 0.3 & 0 \\ 0 & 0 & 0 & 0 & 1.0 \\ 0 & 0 & 0 & 1.0 & 0 \end{pmatrix} \tag{16.3.5}$$

corresponding to the graph



$\mathbf{A}'^T$ has two eigenvectors with unit eigenvalue (you can guess them from the graph): $(0.5, 0.5, 0, 0, 0)$ and $(0, 0, 0, 0.5, 0.5)$. However, $\mathbf{A}'^T$ to the power 2^{32} does not have

these as any of its columns, but is rather

$$(\mathbf{A}'^T)^{2^{32}} = \begin{pmatrix} 1.0 & 0 & 0.7 & 0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0.3 & 0 & 1.0 \end{pmatrix} \tag{16.3.6}$$

thus showing that the model has only unstable equilibria. (The little identity matrix blocks are merely artifacts of the limit cycles having period 2, while 2^{32} is even. In general, you will get some other pattern.)

Successive squaring is a poor way to get the equilibrium distribution for a model that is known (or guessed) to have a single stable equilibrium. A better way, since we already know the eigenvalue, is inverse iteration. Just solve the equation

$$(\mathbf{A}^T - \mathbf{1})\,\mathbf{s}_e = \mathbf{b} \tag{16.3.7}$$

by LU decomposition (§2.3), with the right-hand vector $\mathbf{b} = (1, 1, \ldots, 1)$. If your solver complains about the zero pivot instead of substituting a small value for it (which is what we want for this application), then use the matrix $(\mathbf{A}^T - 0.999999 \times \mathbf{1})$ instead. In either case, you will want to renormalize $\mathbf{s}_e$, to make, e.g., its components have unit sum.

You can test for multiple equilibria by perturbing the right-hand side vector and seeing if you get the same $\mathbf{s}_e$. If you do have multiple equilibria, it is probably time to turn to the methods of Chapter 11 and calculate $\mathbf{A}^T$'s eigenvalues and eigenvectors directly.

That is all (in fact, more) than we want to tell about Markov models in general. We turn now to the real business at hand, which is to estimate statistically the state of a "hidden" Markov model, given only partial or imperfect information.

16.3.1 Hidden Markov Models

In a *hidden Markov model* (HMM), we don't get to observe the state of the model directly. Rather, whenever it is in any state i (one of M states), it emits a *symbol* k, chosen probabilistically from a set of K symbols. The probability of emitting symbol number k from state number i is denoted by

$$b_i(k) \equiv P(\text{symbol } k \mid \text{state } i) \qquad (0 \le i < M, \quad 0 \le k < K) \tag{16.3.8}$$

with the normalization condition

$$\sum_{k=0}^{K-1} b_i(k) = 1 \qquad (0 \le i < M) \tag{16.3.9}$$

Thus, when the model evolves through N timesteps, the *hidden states* are a vector of integers,

$$\mathbf{s} = \{s_t\} = (s_0, s_1, \ldots s_{N-1}) \tag{16.3.10}$$

each in the range $0 \le s_i < M$, while the *observations* or *data* are a vector of integers,

$$\mathbf{y} = \{y_t\} = (y_0, y_1, \ldots y_{N-1}) \tag{16.3.11}$$

each in the range $0 \le y_i < K$.

For the Teen Life example, here is a table of symbols and their probabilities of being emitted from each state, in response to the repeated parental query, "What are you doing?":

			$i = 0$	1	2	3	4
k	symbol	meaning	Eat	Hang	Study	Talk	Sleep
0	o	[silence]	0.2	0.2	0	0.3	0.5
1	s	"I'm studying!"	0	0	1.0	0.2	0
2	b	"I'm busy!"	0	0.6	0	0.4	0
3	g	[grunt]	0.8	0.2	0	0.1	0
4	z	[snore]	0	0	0	0	0.5

The key point is that the emitted symbols give only incomplete, or garbled, state information (e.g., the claim of studying when actually talking on the phone). A state can emit more than one possible symbol, and a symbol can be emitted by more than one possible state. Nevertheless, our goal is to make the best statistical reconstruction of the vector $\mathbf{s}$ from $\mathbf{y}$.

More specifically, at each time t we want to estimate

$$P_t(i) \equiv P(s_t = i \mid \mathbf{y}) \tag{16.3.12}$$

the probability that the actual state of the system is i at time t, given all the data. (If the word "probability" in this context bothers you, you may think of it as a likelihood.)

Let $\alpha_t(i)$ be defined for $t = 0 \ldots N-1$ and $i = 0 \ldots M-1$ as the probability of the observed data up to time t (that is, $y_0 \ldots y_t$), *given* that we are in state i at time t. Since we are not specifying any of the previous states, we must sum over all possible paths that get to state i at time t. Thus,

$$\begin{aligned} \alpha_0(i) &= b_i(y_0) \\ \alpha_t(i) &= \sum_{i_0, i_i, \ldots, i_{t-1}} b_{i_0}(y_0)\, A_{i_0 i_1} b_{i_1}(y_1) \;\ldots\; A_{i_{t-1} i} b_i(y_t) \qquad (1 \le t < N) \end{aligned} \tag{16.3.13}$$

In other words, each transition contributes to the product both a transition probability and a symbol probability, and we sum over all possible combinations of previous states, that is, all possible values of $i_0, i_1, \ldots, i_{t-1}$, each in the range $0 \ldots M-1$.

Since $\alpha_t(i)$ is the probability of data given state, it can also be interpreted as the likelihood of the state, given the data; or, if we are Bayesians (and we are!), as the unnormalized posterior probability of being in state i, which can be normalized simply by dividing by $\sum_i \alpha_t(i)$. However, equation (16.3.13) seems useless for a big and a little reason. Big: It has exponentially many terms to evaluate. Little: it uses only part of the data (the data earlier in time) to estimate the state i at time t. It is what is called a *forward estimate*.

Amazingly, both problems are easy to fix. It is not hard to see that the $\alpha_t(i)$'s

satisfy a recurrence relation that advances them all one step in t:

$$\alpha_{t+1}(j) = \sum_{i=0}^{M-1} \alpha_t(i) A_{ij} b_j(y_{t+1}) \qquad (0 \le j < M) \quad \text{for} \quad t = 0, \ldots, N-2 \tag{16.3.14}$$

One step of this recurrence takes only $O(M^2)$ operations, so the whole table of $\alpha_t(i)$'s can be computed in $O(NM^2)$.

To fix the second problem, we come at the issue from "the other end of time." Let $\beta_t(i)$ be defined for $t = N-1 \ldots 0$ and $i = 0 \ldots M-1$ as the probability of the *future* observed data $(y_{t+1}, y_{t+2}, \ldots y_{N-1})$ again *given* that we are in state i at time t. Analogously to the α's, we have

$$\beta_t(i) = \sum_{i_{t+1}, \ldots, i_{N-1}} A_{i i_{t+1}} b_{i_{t+1}}(y_{t+1}) \; \ldots \; A_{i_{N-2} i_{N-1}} b_{i_{N-1}}(y_{N-1}) \tag{16.3.15}$$

with the special case $\beta_{N-1}(i) = 1$. (Because there are no data to the future of $t = N-1$, those data's probability is by definition 1.) In the formula for the $\beta_t(i)$'s there is a factor in the product for each future transition probability and each future symbol probability (fixing the symbols to be the actual y's of course). Just as for the α's, the β's can be interpreted as likelihoods, or unnormalized posterior probabilities. And, wonderfully, equation (16.3.15) can be solved by a *backward* recurrence,

$$\beta_{t-1}(i) = \sum_{j=0}^{M-1} A_{ij} b_j(y_t) \beta_t(j) \qquad (0 \le i < M) \quad \text{for} \quad t = N-1, \ldots, 1 \tag{16.3.16}$$

Calculating all the β's for $t = N-1, N-2, \ldots, 0$ is called a *backward estimate*.

Now here is the big payoff: From the definitions of the α's and β's, the product $\alpha_t(i)\beta_t(i)$ is the unnormalized posterior probability of state i at time t given *all* the data. If we normalize it by dividing by

$$\mathcal{L}_t = \sum_{i=0}^{M-1} \alpha_t(i) \beta_t(i) \tag{16.3.17}$$

we get the desired estimate of the probability of each separate state at each separate time,

$$P_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\mathcal{L}_t} \tag{16.3.18}$$

Further, it follows from the definitions (16.3.13) and (16.3.15) that $\mathcal{L}_t$ is actually independent of t, so we can omit the subscript t and calculate it only once. (In practice, one often renormalizes at each time t for greater numerical stability, however.) The value of $\mathcal{L}$ can be interpreted as the probability (or likelihood) of the whole data set, given the parameters of the model.

Equations (16.3.14) and (16.3.16), taken together, are called the *forward-backward algorithm* for state estimation in a hidden Markov model.

Translating HMM into code, we start with a structure that will hold the various quantities that come into play, and its constructor. You construct an `HMM` structure by specifying a transition probability matrix $\mathbf{A}$ (N.B.: not $\mathbf{A}^T$), a symbol probability matrix $b_{ik} \equiv b_i(k)$, and a vector of observed data $\mathbf{y}$.

hmm.h

```
struct HMM {
```
Structure for a hidden Markov model and its methods.
```
    MatDoub a, b;                          Transition matrix and symbol probability matrix.
    VecInt obs;                            Observed data.
    Int fbdone;
    Int mstat, nobs, ksym;                 Number of states, observations, and symbols.
    Int lrnrm;
    MatDoub alpha, beta, pstate;           Matrices α, β, and P_i(t).
    VecInt arnrm, brnrm;
    Doub BIG, BIGI, lhood;
    HMM(MatDoub_I &aa, MatDoub_I &bb, VecInt_I &obs);    Constructor; see below.
    void forwardbackward();                HMM state estimation.
    void baumwelch();                      HMM parameter re-estimation.
    Doub loglikelihood() {return log(lhood)+lrnrm*log(BIGI);}
```
Returns the log-likelihood computed by `forwardbackward()`.
```
};

HMM::HMM(MatDoub_I &aa, MatDoub_I &bb, VecInt_I &obss) :
    a(aa), b(bb), obs(obss), fbdone(0),
    mstat(a.nrows()), nobs(obs.size()), ksym(b.ncols()),
    alpha(nobs,mstat), beta(nobs,mstat), pstate(nobs,mstat),
    arnrm(nobs), brnrm(nobs), BIG(1.e20), BIGI(1./BIG)  {
```
Constructor. Input are the transition matrix `aa`, the symbol probability matrix `bb`, and the observed vector of symbols `obss`. Local copies are made, so the input quantities need not be preserved by the calling program.
```
    Int i,j,k;
    Doub sum;
```
Although space constraints make us generally stingy about printing code for checking input, we will save you a lot of grief by doing so in this case. If you get "matrix not normalized" errors, you probably have your matrix transposed. Note that normalization errors <1% are silently fixed.
```
    if (a.ncols() != mstat) throw("transition matrix not square");
    if (b.nrows() != mstat) throw("symbol prob matrix wrong size");
    for (i=0; i<nobs; i++) {
        if (obs[i] < 0 || obs[i] >= ksym) throw("bad data in obs");
    }
    for (i=0; i<mstat; i++) {
        sum = 0.;
        for (j=0; j<mstat; j++) sum += a[i][j];
        if (abs(sum - 1.) > 0.01) throw("transition matrix not normalized");
        for (j=0; j<mstat; j++) a[i][j] /= sum;
    }
    for (i=0; i<mstat; i++) {
        sum = 0.;
        for (k=0; k<ksym; k++) sum += b[i][k];
        if (abs(sum - 1.) > 0.01) throw("symbol prob matrix not normalized");
        for (k=0; k<ksym; k++) b[i][k] /= sum;
    }
}
```

Now, to actually do the forward-backward estimation, you call the function `forwardbackward`. This fills the matrix `pstate`, so that $\texttt{pstate}_{ti} = P_t(i)$. It also sets the internal variables `lhood` and `lrnrm` so that the function `loglikelihood` returns the logarithm of $\mathcal{L}$. Don't be surprised at how large in magnitude this (negative) number can be. The probability of any *particular* data set of more than trivial length is astronomically small!

In the following code, the quantities `BIG`, `BIGI`, `arnrm`, `brnrm`, and `lrnrm` all relate to dealing with values that would far underflow an ordinary floating format. The basic idea is to renormalize as necessary, keeping track of the accumulated number of renormalizations. At the end, when an α, a β, and an $\mathcal{L}$ are combined, probability values of reasonable magnitude result.

```
void HMM::forwardbackward() {                                          hmm.h
```
HMM forward-backward algorithm. Using the stored a, b, and obs matrices, the matrices alpha, beta, and pstate are calculated. The latter is the state estimation of the model, given the data.
```
    Int i,j,t;
    Doub sum,asum,bsum;
    for (i=0; i<mstat; i++) alpha[0][i] = b[i][obs[0]];
    arnrm[0] = 0;
    for (t=1; t<nobs; t++) {                    Forward pass.
        asum = 0;
        for (j=0; j<mstat; j++) {
            sum = 0.;
            for (i=0; i<mstat; i++) sum += alpha[t-1][i]*a[i][j]*b[j][obs[t]];
            alpha[t][j] = sum;
            asum += sum;
        }
        arnrm[t] = arnrm[t-1];                  Renormalize the α's as necessary to avoid under-
        if (asum < BIGI) {                          flow, keeping track of how many renormal-
            ++arnrm[t];                             izations for each α.
            for (j=0; j<mstat; j++) alpha[t][j] *= BIG;
        }
    }
    for (i=0; i<mstat; i++) beta[nobs-1][i] = 1.;
    brnrm[nobs-1] = 0;
    for (t=nobs-2; t>=0; t--) {                 Backward pass.
        bsum = 0.;
        for (i=0; i<mstat; i++) {
            sum = 0.;
            for (j=0; j<mstat; j++) sum += a[i][j]*b[j][obs[t+1]]*beta[t+1][j];
            beta[t][i] = sum;
            bsum += sum;
        }
        brnrm[t] = brnrm[t+1];
        if (bsum < BIGI) {                      Similarly, renormalize the β's as necessary.
            ++brnrm[t];
            for (j=0; j<mstat; j++) beta[t][j] *= BIG;
        }
    }
    lhood = 0.;                                 Overall likelihood is lhood with lnorm renormal-
    for (i=0; i<mstat; i++) lhood += alpha[0][i]*beta[0][i];        izations.
    lrnrm = arnrm[0] + brnrm[0];
    while (lhood < BIGI) {lhood *= BIG; lrnrm++;}
    for (t=0; t<nobs; t++) {                    Get state probabilities from α's and β's.
        sum = 0.;
        for (i=0; i<mstat; i++) sum += (pstate[t][i] = alpha[t][i]*beta[t][i]);
        The next line is an equivalent calculation of sum. But we'd rather have the normaliza-
        tion of the P_i(t)'s be more immune to roundoff error. Hence we do the above sum for
        each value of t.
        // sum = lhood*pow(BIGI, lrnrm - arnrm[t] - brnrm[t]);
        for (i=0; i<mstat; i++) pstate[t][i] /= sum;
    }
    fbdone = 1;                                 Flag prevents misuse of baumwelch(), later.
}
```

You may be wondering how well `forwardbackward` is able to do at predicting the hidden states of Teen Life, given just a long string of output symbols. If we take the prediction to be the state with the highest probability at each time, then this is correct about 78% of the time. Another 17% of the time, the correct state has the second-highest probability, often when the top two probabilities are nearly equal. It is an important property of HMMs that the output is not only a prediction, but also a quantitative assessment of how "sure" the model is of that prediction.

16.3.2 Some Variations on HMM

HMM state estimation with the forward-backward algorithm is a very flexible formalism, and many variants are possible. For example, in decoding codes on a trellis, as we did above, the symbols 0 or 1 are emitted not by the states, but by the transitions between the states. If we want to use HMM for that problem (we will say more about this below), we must replace $b_i(k)$ with $b_{ij}(k)$, the probability of emitting symbol k in a transition from state i to state j. The forward and backward recurrences now become

$$\begin{aligned}\alpha_{t+1}(j) &= \sum_{i=0}^{M-1} \alpha_t(i) A_{ij} b_{ij}(y_{t+1}) \\ \beta_{t-1}(i) &= \sum_{j=0}^{M-1} A_{ij} b_{ij}(y_t) \beta_t(j)\end{aligned} \tag{16.3.19}$$

and we start off the α's with the special rule $\alpha_0(i) = 1$, since (like the case of $\beta_{N-1}(i)$ previously) the probability of the data is 1 before there are any data.

Another variant case is where one or more intermediate states are known exactly. In that case, one or more of the sums over $i_0, i_1, \ldots, i_{t-1}$ in equation (16.3.13) is left out, and the corresponding index on an A and b gets replaced by the known state number. If you track through how this affects the recurrence equation (16.3.14), you'll see that the new procedure is

- calculate the α's forward to, and including, a known state;
- zero all the α values at that time except for that of the known state;
- don't renormalize anything (though you feel tempted to do so); and
- continue forward with the α's for the next timestep.

Proceed similarly for the β's.

The opposite variant is where you have *missing data*, meaning that for some values of t there is no observation of the symbol $\mathbf{y}_t$. In this case, all you need to do is to make a special case for the symbol probability,

$$b_i(y_t) \equiv 1, \qquad (0 \le i < M) \qquad t \in \{\text{missing}\} \tag{16.3.20}$$

meaning that, regardless of state i, the probability of observing the data (meaning no data) at time t is unity. Now proceed as usual to calculate the state probabilities. If you then want to *reconstruct* the missing data, you can calculate its posterior probabilities,

$$P(y_t = k \mid \mathbf{y}) = \sum_{i=0}^{M-1} P_i(t) b_i(k) = \sum_{i=0}^{M-1} \frac{\alpha_i(t)\beta_i(t)}{\mathcal{L}} b_i(k) \qquad t \in \{\text{missing}\} \tag{16.3.21}$$

16.3.3 Bayesian Re-Estimation of the Model Parameters

This is magical. The probability that we were in state i at time t is $\alpha_t(i)\beta_t(i)/\mathcal{L}$. What is the probability, given the data $\mathbf{y}$, that a given transition, say between time

t and time $t+1$, was a transition between state i and state j? We write, applying various of the laws of probability,

$$\begin{aligned} P(s_t = i, s_{t+1} = j \mid \mathbf{y}) &= P(s_{t+1} = j \mid \mathbf{y}, s_t = i) P(s_t = i \mid \mathbf{y}) \\ &= \frac{P(\mathbf{y} \mid s_{t+1} = j, s_t = i) P(s_{t+1} = j \mid s_t = i)}{\sum_j P(\mathbf{y} \mid s_{t+1} = j, s_t = i) P(s_{t+1} = j \mid s_t = i)} P(s_t = i \mid \mathbf{y}) \\ &= \frac{[\alpha_t(i) b_j(y_{t+1}) \beta_{t+1}(j)][A_{ij}]}{\sum_j [\alpha_t(i) b_j(y_{t+1}) \beta_{t+1}(j)][A_{ij}]} \frac{[\alpha_t(i)\beta_t(i)]}{\mathcal{L}} \\ &= \frac{\alpha_t(i) A_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\mathcal{L}} \end{aligned} \tag{16.3.22}$$

Note how the sum over j in the denominator disappears by the recurrence (16.3.16) for $\beta_t(i)$.

So, for a long run of data, we can compute the fraction of the time that a state i transitions to state j as the estimated number of $i \to j$ transitions divided by the estimated number of i states,

$$\widehat{A}_{ij} = \frac{\sum_t \alpha_t(i) A_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\sum_t \alpha_t(i) \beta_t(i)} \tag{16.3.23}$$

noting that the $\mathcal{L}$'s cancel out. The reason for calling this quotient $\widehat{A}_{ij}$ is that it is a *re-estimation* of the transition probability A_{ij}. The corresponding re-estimation of the symbol probability matrix $b_i(k)$ is the fraction of all i states that emit a symbol k, namely

$$\widehat{b}_i(k) = \frac{\sum_t \delta(y_t, k) \alpha_t(i) \beta_t(i)}{\sum_t \alpha_t(i) \beta_t(i)} \tag{16.3.24}$$

where $\delta(j,k)$ is 1 if $j = k$, zero otherwise.

You might think that this process is somehow circular, or that re-estimating A_{ij} and $b_i(k)$ in this fashion only introduces noise that degrades the model. Far from it! Baum and Welch first showed that replacing A_{ij} by $\widehat{A}_{ij}$ and $b_j(k)$ by $\widehat{b}_j(k)$, and then recalculating the probabilities of each state at each time by the forward-backward algorithm, always *increases* $\mathcal{L}$, the overall likelihood of the model. It is, in fact, an EM algorithm (cf. §16.1, and see below). You can continue this cycle of estimating states (forward-backward) and re-estimating model probabilities (Baum-Welch), obtaining further increases in $\mathcal{L}$, until convergence to a maximum is achieved. Equations (16.3.23) and (16.3.24) are known as *Baum-Welch re-estimation*.

So the magic is this: We began by estimating states in a *known* hidden Markov model. We now see that, *just from the data*, we can get not only an estimate of the states, but also an estimate of the model itself, that is, the transition probabilities and symbol probabilities. Like any iterative process, this works best if we have a good initial guess. But it will often converge to a good model from a fairly random initial guess. (You should not start with exactly uniform probabilities, because that creates a symmetry that the iteration finds hard to break.)

The code is straightforward. The updating of $b_i(k)$ comes almost for free as a byproduct of computing the denominator in the update for A_{ij}. Like the forward-backward algorithm, Baum-Welch re-estimation takes $O(NM^2)$ operations.

hmm.h

```
void HMM::baumwelch() {
```

Baum-Welch re-estimation of the stored matrices a and b, using the data obs and the matrices alpha and beta as computed by forwardbackward() (which must be called first). The previous values of a and b are overwritten.

```
    Int i,j,k,t;
    Doub num,denom,term;
    MatDoub bnew(mstat,ksym);
    Doub powtab[10];                    Fill table of powers of BIGI.
    for (i=0; i<10; i++) powtab[i] = pow(BIGI,i-6);
    if (fbdone != 1) throw("must do forwardbackward first");
    for (i=0; i<mstat; i++) {           Looping over i, get denominators and new b.
        denom = 0.;
        for (k=0; k<ksym; k++) bnew[i][k] = 0.;
        for (t=0; t<nobs-1; t++) {
            term = (alpha[t][i]*beta[t][i]/lhood)
                * powtab[arnrm[t] + brnrm[t] - lrnrm + 6];
            denom += term;
            bnew[i][obs[t]] += term;
        }
        for (j=0; j<mstat; j++) {       Inner loop over j gets elements of a.
            num = 0.;
            for (t=0; t<nobs-1; t++) {
                num += alpha[t][i]*b[j][obs[t+1]]*beta[t+1][j]
                    * powtab[arnrm[t] + brnrm[t+1] - lrnrm + 6]/lhood;
            }
            a[i][j] *= (num/denom);
        }
        for (k=0; k<ksym; k++) bnew[i][k] /= denom;
    }
    b = bnew;
    fbdone = 0;                         Don't let this routine be called again until forward-
}                                           backward() has been called.
```

You must always precede a call to `baumwelch` by a call to `forwardbackward`, since the latter updates the α and β tables. Also, as you alternate calls to the two functions, you monitor convergence by the value of the log-likelihood calculated by `forwardbackward`.

Be aware that convergence can be excruciatingly slow! The references describe methods by which convergence can be accelerated in some cases. Common difficulties are when a rare state is not correctly captured by the model, or when the model thinks that there are two states, with nearly identical transition probabilities, when there is really only one. If you have even a plausible guess for the transition probability matrix, you should use it to start. There are many applications where you shouldn't use re-estimation at all: If you have a pretty good model to start with, just use it (via `forwardbackward`) to analyze your data, and don't even think about re-estimating.

The Baum-Welch re-estimation algorithm, which dates from the mid-1960s, was generalized in the mid-1970s by Dempster, Laird, and Rubin, as the *expectation-maximization (EM) algorithm*, with a variety of applications to problems with missing or censored data. (An example is the Gaussian mixture model in §16.1.) In this more general language, the forward-backward algorithm is an E-step, while Baum-Welch is an M-step. Alas, one small cloud in an otherwise bright sky is that the maximum of $\mathcal{L}$ achieved by multiple EM iterations is only guaranteed to be a local, not a global, maximum.

HMM has found wide application in speech recognition, gene sequence comparison, financial models, and a variety of other fields. The references give specifics.

16.3.4 Comparing the Viterbi Algorithm with HMM

It is important to understand the similarities and differences between the Viterbi algorithm and hidden Markov modeling (its forward-backward algorithm in particular).

When we discussed the Viterbi algorithm in the context of decoding, we made the implicit assumption that a 1 bit was a priori as likely as a 0 bit. It is straightforward to generalize the Viterbi algorithm to include an arbitrary a priori transition probability A_{ij}, just like HMM. In that case, the probability factor on each edge (whose negative logarithm is the edge cost) is the product of two terms, again just like HMM, $A_{ij}b_{ij}(k)$, where now $b_{ij}(k)$ is the probability of observing the observed symbol k given that an $i \to j$ transition occurred.

We discussed Baum-Welch parameter re-estimation for HMMs in some detail. Re-estimation of the parameters in a Viterbi model, often called *Viterbi training*, is analogous. Take the most probable path output by the algorithm (or ensemble of paths collected from the decodes of many codewords). Count the number of $i \to j$ transitions seen along these paths and the numbers of each symbol k seen for each pair i, j. Now re-estimate A_{ij} and $b_{ij}(k)$ by the obvious normalizations of these counts.

The Viterbi algorithm and the forward estimation part of the forward-backward algorithm are structurally very similar. In both cases, we sweep forward in time (or by stages) and assign a likelihood (or posterior probability) to each node, based on the data already seen. The difference is that Viterbi assigns to a node the probability of the *single best path* that reaches it, while forward-backward assigns the sum of probabilities over *all possible paths* to that node. Indeed the Viterbi algorithm is sometimes called the *min-sum algorithm* while forward-backward is referred to as the *sum-product algorithm*, just to highlight this distinction. (In the context of coding theory, the forward-backward algorithm is also sometimes called the *BCJR* or *Bahl-Cocke-Jelinek-Raviv* algorithm. In other contexts it is sometimes called *belief propagation*.)

The backward passes of the two algorithms have somewhat different structures. For Viterbi, the backward pass simply consolidates the information about the single most probable path that is already implicit in the node labeling. For forward-backward, as we have seen, the backward pass is needed to get posterior probabilities for each node that use all the data, both ahead of and behind any time t.

If you think you have a choice between using the Viterbi algorithm or using HMM, you should probably think again. Most problems clearly favor only one or the other method. If your desired output is a valid *path* on the graph, then HMM won't do: It might yield a set of highly probable nodes that just don't lie on any single path. For example, you might have the first half of one codeword and the second half of another, with no graph edge connecting the two halves. That is why decoding theory usually starts in the world of Viterbi (although, in some more complicated constructs, it can end up with a foot in each world).

On the other hand, if you care about which *nodes* are visited, then HMM is most likely what you want. In fact, Viterbi can give very poor results. The most probable path is often *very improbable* when compared to the sum of all paths that lead through a particular node, one possibly not on the most probable path. Or, another way of describing this, there may be exponentially many paths with not-too-different probabilities, so the node probabilities are determined by the statistics of where they

all channel, not by which one path happens to have the highest probability.

It is very easy to "mine" HMM for alternative possibilities, since it yields seemingly every possible posterior probability that you might want to know. It is quite difficult to get anything from the Viterbi algorithm other than the single most probable path. That is because the enumeration of all possible paths is vastly harder than the enumeration of all possible nodes; the Bellman-Dijkstra-Viterbi algorithm is exquisitely good at keeping only the information that it needs. Data structures for finding more than one probable path rapidly become very complex.

Finally, we must take aim at the myth, occasionally heard, that the Viterbi algorithm, as a pure maximum likelihood (ML) estimate, is somehow "less Bayesian" than HMM. In fact, HMM is also a pure ML estimate if you look only at the state i with the largest $\alpha_t(i)\beta_t(i)$ at each time t, neither normalizing its value nor looking at any other i's. But you are then ignoring a wealth of useful information about the other possible states. (This, in part, is why you should get with the Bayesian program!) We think that both HMM and Viterbi are in fact Bayesian to the core. If there were good ways to enumerate all the other paths and their likelihoods, we would not hesitate to normalize the best-path likelihood and call it a posterior probability. It is only because of the difficulty of this enumeration that it is possible to keep the Viterbi algorithm's Bayesian character "in the closet"; and there is no advantage, that we can see, in doing so.

CITED REFERENCES AND FURTHER READING:

Hsu, H.P. 1997, *Schaum's Outline of Theory and Problems of Probability, Random Variables, and Random Processes* (New York: McGraw-Hill).

Häggström, O. 2002, *Finite Markov Chains and Algorithmic Applications* (Cambridge, UK: Cambridge University Press).

Norris, J.R. 1998, *Markov Chains*, Cambridge Series in Statistical and Probabilistic Mathematics (Cambridge, UK: Cambridge University Press).

MacDonald, I.L. and Zucchini, W. 1997, *Hidden Markov and Other Models for Discrete-Valued Time Series* (Boca Raton, FL: Chapman & Hall/CRC).

McLachlan, G.J. and Krishnan, T. 1996, *The EM Algorithm and Extensions* (New York: Wiley).

Rabiner, L. 1989, "A Tutorial on Hidden Harkov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257-286. [Review article on the use of HMMs in speech recognition.]

Eddy, S.R. 1998, "Profile Hidden Markov Models," *Bioinformatics*, vol. 14, pp. 755-763. [Review article on the use of HMMs in genetics.]

16.4 Hierarchical Clustering by Phylogenetic Trees

Hierarchical clustering is a type of *unsupervised learning*: We seek algorithms that figure out how to cluster an unordered set of input data without ever being given any training data with the "right answer." As the name implies, the output of a hierarchical clustering algorithm is a bunch of fully nested sets. The smallest sets are the individual data elements. The largest set is the whole data set. Intermediate

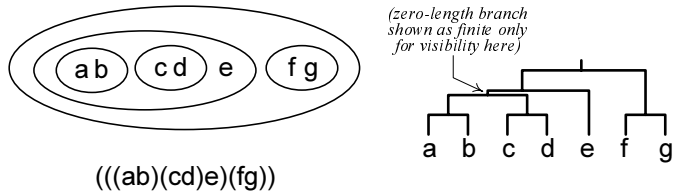


Figure 16.4.1. Representations of hierarchical classification. Top left: Diagram showing fully nested sets. Bottom left: Equivalent parenthesized expression. Right: Binary tree (with possibly zero branch lengths).

sets are nested; that is, the intersection of any two sets is either the null set, or else the smaller of the two sets.

What you need to get started with hierarchical clustering is either a set of *sequences*, or else a *distance matrix*. *Character-based methods* start with n sequences each m characters long (for example, DNA bases or protein amino acids). A toy example might be the $n = 16$ sequences of $m = 12$ characters,

```
0.  CGGTTGGGAGCT      8.  GCGCGGTGCAGC
1.  AGGTCGTGAGGT      9.  AGGCGGTGCGGG
2.  TGGTTGGGGTTT     10.  GGGCGGGGCGGG
3.  TGGGTGCGAGTT     11.  GGGCGCTGCGGG
4.  ACGTTTGGGTGA     12.  GGACGGAGGCTG
5.  AAGGTTGGGGAA     13.  GGGTGGGAGCTG
6.  GTCTTTCGGGTG     14.  AGGAGGCTGATG
7.  CACTTGCGGGGG     15.  TGGCGGATGATG
```

It is probably not immediately obvious that these sequences were generated from a balanced five-level binary tree, with GGGGGGGGGGGG at the root, and with each daughter node having two random mutations from her parent. We will see below the extent to which some of the algorithms that we discuss can figure this out from the data. A realistic case likely would have significantly longer sequences than this, and fewer mean mutations per character; the number of sequences might be either more, or fewer, than this toy.

The alternative starting point is with an $n \times n$ matrix d_{ij} of distances between all pairs of your n data points, which might now be sequences, points in N-dimensional space, or whatever. You are responsible for making sure that the distance matrix satisfies four conditions:

$$\begin{aligned} d_{ij} &\geq 0 && \text{(positivity)} \\ d_{ii} &= 0 && \text{(zero self-distance)} \\ d_{ij} &= d_{ji} && \text{(symmetry)} \\ d_{ik} &\leq d_{ij} + d_{jk} && \text{(triangle inequality)} \end{aligned} \tag{16.4.1}$$

for all i, j, k. We'll discuss below how to get distances from sequences, if that's the way you want to go.

Figure 16.4.1 shows three representations of the same hierarchical clustering of seven data elements. The two representations on the left are self-explanatory. The one on the right, the binary tree, takes explaining on one point: If, in the set diagram, (ab), (cd), and (e) are clustered "democratically," then why does the binary tree

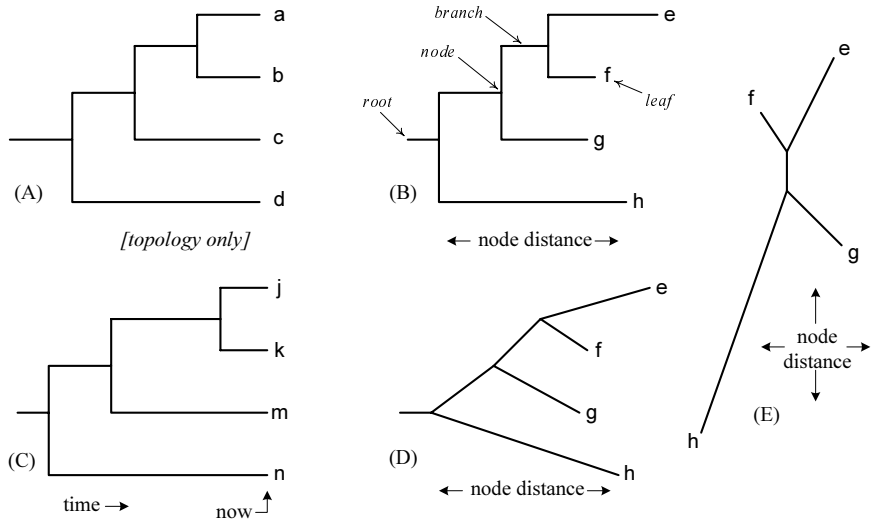


Figure 16.4.2. Types of trees. A *cladogram* (A) has arbitrary branch lengths. Only the topology is intended to be represented. A *phylogram* or *phylogenetic tree* (B) is an additive tree, where the distance between any two nodes/leaves is the sum of lengths of the horizontal connecting branches. An *ultrametric tree* (C) is an additive tree with the property that any node has the same distance to all of its leaves (as when all lengths represent time). Tree (D) is an alternative way of drawing tree (B). Again, only horizontal distances are significant. In an *unrooted tree* (E), line lengths represent distances independent of orientation. The tree (E) is the unrooted depiction of (B) or (D).

select (e) arbitrarily as the descendant of a higher node, instead of having three equal descendants from a common node?

The answer is just convention. Binary trees are the adopted common language of hierarchical clustering because (i) they emerge naturally from the concept of mutation events in biology, (ii) they are somewhat easier to represent in a computer than more general trees, and, (iii) they are often easier to prove theorems about. Our binary trees will almost always have the concept of *branch length*, a representation of some measure of difference between a parent node and its child. When we need to connect up democratically some number of nodes greater than two, we do it with zero-length branches. A convention is to view all topologies of nodes so connected as being equivalent.

16.4.1 Tree Basics

Figure 16.4.2 shows several ways of drawing binary trees and introduces some further terminology. The data points are *leaves*, meaning that they are generally taken as terminal nodes on the tree. Trees are often drawn either sideways (root left, leaves right) or upside down (root top, leaves bottom) by comparison with their arboreal namesake whose roots are on the bottom, leaves on top — at least for most trees that we see! A tree without meaningful branch lengths is usually called a *cladogram*. These have a rich historical tradition in pre-molecular biology but are viewed with alarm by most bioscientists today. A tree with meaningful branch lengths representing *distances* (in some metric) between nodes and their children, or between leaves, is called a *phylogram* or *phylogenetic tree*. (However, some authors use the terms cladogram and phylogram interchangeably, while some others use the words merely

to distinguish different drawing styles.)

To a mathematician, a phylogenetic tree is an *additive tree*, meaning that the tree's path lengths induce a *distance metric* between any two leaves, namely the sum of path lengths up and down that connect the two leaves through their unique closest common ancestor. In real situations, the data we are given often are not exactly represented by an additive distance metric. Thus, the problem of hierarchical clustering amounts to finding a way of projecting such data onto the set of all additive trees in a useful, or statistically justifiable, way.

Given an additive tree, it is easy to compute its distance matrix d_{ij}, defined now as the matrix of all distances between pairs of leaf nodes. But what about the reverse? Given some symmetric matrix d_{ij}, is it possible to determine whether there exists an additive tree that instantiates it? Yes. One answer is the *four-point condition* for additive trees: Given four distinct leaves i, j, k, l, an additive tree exists if and only if

$$d_{ij} + d_{kl} \leq \max(d_{ik} + d_{jl},\ d_{il} + d_{jk}) \tag{16.4.2}$$

for all choices of i, j, k, l. In words, this is equivalent to the statement: For all distinct i, j, k, l, there is a tie for the maximum of the three sums of the form $d_{ij} + d_{kl}$. Later, when we discuss the neighbor-joining method, we will have a more practical algorithmic answer.

As Figure 16.4.2 illustrates, a tree can either be *rooted* or *unrooted*. An unrooted tree can always be rooted arbitrarily, by choosing any branch, grasping its midpoint between your thumb and forefinger, and then shaking the tree so that all the other branches drop downward into place. (We could give a much more mathematical description, but it would not add any clarity.) Some hierarchical clustering algorithms yield rooted trees, where the root has some meaning with respect to the data; others yield unrooted trees, although they may be drawn as if rooted, simply as a graphical convention. It's important to keep track of which kind of algorithm you are using.

You may wonder why the data points must always be leaves (terminal nodes). Might not some data points actually be good ancestors of others? The answer is again a mixture of history and convention: If the leaves are observed living taxa, then they are by definition alive today. If "today" represents terminal nodes, then they are by definition leaves. What makes this a mere convention is that we can always connect a leaf to an ancestor node by a zero-length branch, so that ancestor-versus-living-taxon becomes a distinction without a difference. A benefit of this convention is that we always know in advance how many internal nodes will be generated by n data points: $n - 1$ if the tree is rooted, or $n - 2$ for an unrooted tree, independent of the tree's topology. (If this isn't obvious, then draw a few pictures.)

If path length denotes, literally, evolutionary time, then a phylogenetic tree has the additional property of being *ultrametric* (refer to Figure 16.4.2). Ultrametric trees are defined as additive trees with the property that the distance from any node to all of its descendant leaves is the same for all such leaves. Clearly this is the case if all the leaves have the same "time distance" from their common ancestor. In the early 1960s, it was proposed that accepted mutation rates might be close enough to constant that, at the molecular level, actual evolutionary data might be close to ultrametric, i.e., that there was a "molecular clock." For most cases, this is no longer believed to be true. For example, mutation rates in *E. coli* have been found to vary by two orders of magnitude. Ultrametric trees are important mathematically, as we will see, but almost never realistic in their own right.

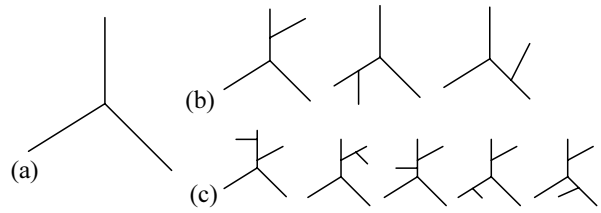


Figure 16.4.3. Tree counting. (a) There is just one unrooted tree with three leaves. (b) There are three ways to add a fourth leaf. (c) For each of the trees in (b), there are five ways to add a fifth leaf. Continuing to add leaves, the number of trees with n leaves is $1 \times 3 \times 5 \times \cdots \times (2n-5)$.

The test, analogous to equation (16.4.2), for whether a given distance matrix is ultrametric is the *three-point condition*,

$$d_{ij} \le \max(d_{ik}, d_{jk}) \tag{16.4.3}$$

for all distinct i, j, k. Equivalently, in words: Among the three distances connecting three distinct leaves, there is a tie for the maximum. Here too there is a more practical algorithmic answer, which we will mention later.

There are a *lot* of possible ways to draw a tree that connects n leaves. As Figure 16.4.3 illustrates, the number of distinct, unrooted, possibilities is

$$N_{\text{trees}}(n) = 1 \times 3 \times 5 \times \cdots \times (2n-5) \equiv (2n-5)!! \tag{16.4.4}$$

The fact that this expression grows super-exponentially with n creates a dilemma in the field of computational phylogenetics: An algorithm that requires an explicit enumeration of, or explicit search over, all possible trees can be useful only for small values of n. Thus, $N_{\text{trees}}(10) \approx 2 \times 10^6$, is easy, but $N_{\text{trees}}(20) \approx 2 \times 10^{20}$ is practically impossible.

16.4.2 Strategies for the Hierarchical Clustering Problem

If you are starting with a set of sequences, then, schematically, the goal of a character-based method is to find the best of all possible trees, given the data, for some definition of "best":

$$\text{(sequences)} \xrightarrow{\substack{\text{search for} \\ \text{"best" tree}}} \text{(tree)} \tag{16.4.5}$$

The two most common definitions of "best" are *maximum parsimony* and *maximum likelihood*, both of which we will say more about below [1]. Character methods are generally limited by the super-exponential explosion in the number of trees. Although the exhaustive search can be limited to some extent, for example by heuristics of various kinds, its long shadow can never be avoided entirely.

Alternatively, if you are starting with a distance matrix, the problem is to find the additive tree whose induced distance matrix (by branch lengths up and down) is closest to d_{ij}, according to some criterion of closeness. This is also an exponentially difficult problem. In practice, however, distance methods almost always use fast heuristic methods that, while provably exact only for the unrealistic case where d_{ij}

already comes from an ultrametric or additive tree, actually work pretty well for distance matrices encountered in practice. In other words, the adopted scheme is

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{heuristic}]{\text{ultrametric tree}} (\text{tree}) \tag{16.4.6}$$

or

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{heuristic}]{\text{additive tree}} (\text{tree}) \tag{16.4.7}$$

The ability of these fast heuristic methods to give "pretty good" solutions to NP-hard problems is remarkable, and only partially understood [2].

The most widely used heuristics are all *agglomerative methods*, meaning that they start by connecting individual data points into small clusters, then connect those clusters, and so forth. Common ultrametric-tree heuristic methods are *UPGMA*, *WPGMA*, *single linkage clustering*, and *complete linkage clustering*. The most widely used additive-tree heuristic method — and probably the most widely used phylogenetic clustering method overall — is the *neighbor-joining (NJ)* method [6]. We will discuss, and implement, all the mentioned heuristic methods below.

There are a few, less-well-developed, distance-based methods that avoid heuristics by finding provably error-bounded methods for transforming an arbitrary distance matrix into the matrix of an additive tree, then exactly constructing the resulting tree [3,4],

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{additive}]{\text{find nearby}} \begin{pmatrix}\text{additive}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{construction}]{\text{exact}} (\text{tree}) \tag{16.4.8}$$

Though more rigorous than the heuristic methods, there is little evidence that these methods produce better results [5].

Evidently, one can always turn a character-based problem into a distance-based one by defining a distance on character sequences,

$$(\text{sequences}) \xrightarrow[\text{distance}]{\text{define a}} \begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \tag{16.4.9}$$

and then continuing with schemes (16.4.6) or (16.4.7).

The obvious distance between two sequences is their *Hamming distance* $H(i,j)$, which is defined as the number of character positions in which sequence i differs from sequence j, an integer between 0 and m. However, when you are given not just the data, but also a statistical model defining how it was generated (i.e., "evolved"), there is often a *corrected distance transformation* that will give better tree reconstructions [2]. For example, the popular *Cavender-Felsenstein* model (whose discussion is beyond our scope) has the corrected distance transformation

$$d_{ij} = -\tfrac{1}{2}\log\left(1 - 2H(i,j)/m\right) \tag{16.4.10}$$

This expression can be used directly when sequences are long enough, or mutation probabilities small enough, that the argument of the logarithm is never negative. If your data produce a negative argument, then a standard workaround is to use a multiple ($1\times$ or $2\times$) of the largest computable d_{ij} for all uncomputable d_{ij}'s. Corrected distance transformations also exist for general Markov models.

Corrected distance transformations have the defining (and desirable) property that as the sequence length increases, the matrix of observed corrected distances will converge to the distance matrix of an additive tree. (This is not true for the uncorrected Hamming distance, incidentally.) In such a case, the power of an additive-tree heuristic method like neighbor joining is much less mysterious. Corrected distance transformations thus provide a statistical justification for the use of the neighbor-joining method.

16.4.3 Implementation of Agglomerative Methods

The general scheme of an agglomerative method is, first, to initialize n active clusters, each containing one data point, and, second, to repeat the following operations exactly $n - 2$ times:

- Find the two active clusters that are closest by some prescribed distance measure.
- Create a new active cluster that combines the two.
- Connect the new cluster, as parent, to the two closest clusters, as children, with some prescription for the two branch lengths.
- Delete the two children from the active list.
- Compute, by some prescription, distances from the new cluster to the active clusters that remain.

Each repetition of these steps reduces the active cluster list by exactly one (one addition, two deletions), so after $n - 2$ repetitions there will be exactly two active clusters. You connect these either by a single branch (unrooted case), or by creating a root node between them (rooted case) with some prescription as to the two root branch lengths.

As we now turn to implementing phylogenetic tree routines, a few words of caution are in order. Hamming's dictum, that the purpose of computing is insight, not numbers, applies here: Much of the value of a phylogenetic tree program lies in its graphics and user interface, both areas outside of our scope. If you are working with any significant quantity of real data, you probably want to use a sophisticated package. As we write, PAUP (Phylogenetic Analysis Using Parsimony) [7] is the most widely used commercial package, both for maximum parsimony trees and also for the various heuristic methods. PHYLIP (Phylogeny Inference Package) [8] is a free package for smaller trees ($\lesssim 20$ taxa). TreeView is a widely used, free, program used for drawing trees in various formats. A user guide to the use of these and other programs is [9]. If the insight you desire lies in algorithmics, not production data, then you may read on.

Here is an abstract base class that implements the general agglomerative method, leaving the various "prescriptions" to be specified by particular derived classes, which we give later.

phylo.h

```
struct Phylagglomnode {
Node for phylogenetic tree.
    Int mo,ldau,rdau,nel;          Pointers up and down; no. of elements.
    Doub modist,dep,seq;           Branch length to parent. See text re. dep and
};                                     seq.

struct Phylagglom{
Abstract base class for constructing an agglomerative phylogenetic tree.
```

```
Int n, root, fsroot;                    No. of data points, root node, forced root.
Doub seqmax, depmax;                    Max. values of seq, dep over the tree.
vector<Phylagglomnode> t;               The tree.
virtual void premin(MatDoub &d, VecInt &nextp) = 0;
```
Function called before minimum search.
```
virtual Doub dminfn(MatDoub &d, Int i, Int j) = 0;
```
Distance function to be minimized
```
virtual Doub dbranchfn(MatDoub &d, Int i, Int j) = 0;
```
Branch length, node i to mother (j is sister).
```
virtual Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) = 0;
```
Distance function for newly constructed nodes.
```
virtual void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
Doub &bi, Doub &bj) = 0;
```
Sets branch lengths to the final root node.
```
Int comancestor(Int leafa, Int leafb);          See text discussion of NJ.
Phylagglom(const MatDoub &dist, Int fsr = -1)
    : n(dist.nrows()), fsroot(fsr), t(2*n-1) {}
```
Constructor is always called by a derived class.

```
void makethetree(const MatDoub &dist) {
```
Routine that actually constructs the tree, called by the constructor of a derived class.
```
    Int i, j, k, imin, jmin, ncurr, node, ntask;
    Doub dd, dmin;
    MatDoub d(dist);                    Matrix d is initialized with dist.
    VecInt tp(n), nextp(n), prevp(n), tasklist(2*n+1);
    VecDoub tmp(n);
    for (i=0;i<n;i++) {                 Initializations on leaf elements.
        nextp[i] = i+1;                 nextp and prevp are for looping on the distance
        prevp[i] = i-1;                     matrix even as it becomes sparse.
        tp[i] = i;                      tp points from a distance matrix row to a tree
        t[i].ldau = t[i].rdau = -1;         element.
        t[i].nel = 1;
    }
    prevp[0] = nextp[n-1] = -1;         Signifying end of loop.
    ncurr = n;
    for (node = n; node < 2*n-2; node++) {      Main loop!
        premin(d,nextp);                Any calculations needed before min finding.
        dmin = 9.99e99;
        for (i=0; i>=0; i=nextp[i]) {               Find i, j pair with min distance.
            if (tp[i] == fsroot) continue;
            for (j=nextp[i]; j>=0; j=nextp[j]) {
                if (tp[j] == fsroot) continue;
                if ((dd = dminfn(d,i,j)) < dmin) {
                    dmin = dd;
                    imin = i; jmin = j;
                }
            }
        }
        i = imin; j = jmin;
        t[tp[i]].mo = t[tp[j]].mo = node;           Now set properties of the parent
        t[tp[i]].modist = dbranchfn(d,i,j);             and children.
        t[tp[j]].modist = dbranchfn(d,j,i);
        t[node].ldau = tp[i];
        t[node].rdau = tp[j];
        t[node].nel = t[tp[i]].nel + t[tp[j]].nel;
        for (k=0; k>=0; k=nextp[k]) {               Get new-node distances.
            tmp[k] = dnewfn(d,k,i,j,t[tp[i]].nel,t[tp[j]].nel);
        }
        for (k=0; k>=0; k=nextp[k]) d[i][k] = d[k][i] = tmp[k];
        tp[i] = node;           New node replaces child i in dist. matrix, while child
        if (prevp[j] >= 0) nextp[prevp[j]] = nextp[j];  j gets patched around.
        if (nextp[j] >= 0) prevp[nextp[j]] = prevp[j];
        ncurr--;
    }                                   End of main loop.
```

```
            i = 0; j = nextp[0];                    Set properties of the root node.
            root = node;
            t[tp[i]].mo = t[tp[j]].mo = t[root].mo = root;
            drootbranchfn(d,i,j,t[tp[i]].nel,t[tp[j]].nel,
                t[tp[i]].modist,t[tp[j]].modist);
            t[root].ldau = tp[i];
            t[root].rdau = tp[j];
            t[root].modist = t[root].dep = 0.;
            t[root].nel = t[tp[i]].nel + t[tp[j]].nel;
            We now traverse the tree computing seq and dep, hints for where to plot nodes in a
            two-dimensional representation. See text.
            ntask = 0;
            seqmax = depmax = 0.;
            tasklist[ntask++] = root;
            while (ntask > 0) {
                i = tasklist[--ntask];
                if (i >= 0) {                       Meaning, process going down the tree.
                    t[i].dep = t[t[i].mo].dep + t[i].modist;
                    if (t[i].dep > depmax) depmax = t[i].dep;
                    if (t[i].ldau < 0) {            A leaf node.
                        t[i].seq = seqmax++;
                    } else {                        Not a leaf node.
                        tasklist[ntask++] = -i-1;
                        tasklist[ntask++] = t[i].ldau;
                        tasklist[ntask++] = t[i].rdau;
                    }
                } else {                            Meaning, process coming up the tree.
                    i = -i-1;
                    t[i].seq = 0.5*(t[t[i].ldau].seq + t[t[i].rdau].seq);
                }
            }
        }
};
```

The `Phylagglom` structure creates a tree of `Phylagglomnodes`. Each node carries pointers to its mother and two daughters, and knows its number of elements (original data points), branch length to its mother, and two floating values `dep` and `seq`, which we now explain: The final `while` block in `makethetree()` does a depth-first traversal of the finished tree. When it reaches a node in the downward direction, it sets `dep` to the sum of branch lengths to the root node. The variable `dep` is thus a hint as to where to plot the node in the depth direction. When the traversal reaches a leaf, it sets `seq` to a sequential numbering of leaves. When it reaches an internal node in the upward direction, it sets its `seq` value to the average of the `seq` values of its two children. The value of `seq` thus becomes a hint as to where to plot a node perpendicular to the depth direction. If you plot nodes by `dep` and `seq`, then plotted branches won't cross each other.

Looking at the nested loops, you can see that `makethetree()` is $O(n^3)$ in time. Actually, it is straightforward to reduce this to $O(n^2)$: With some extra bookkeeping, you can keep the distances in a structure that allows the shortest to be found without an n^2 search. We have not coded this, just to keep the code shorter and simpler.

16.4.4 Algorithms That Are Exact for Ultrametric Trees

Given a distance matrix that is exactly ultrametric, all of the agglomerative algorithms that we now discuss will (modulo some technical details) reconstruct its tree exactly. The reason that we need more than one such algorithm is because their behaviors can be somewhat different on realistic, nonultrametric, data, in the general

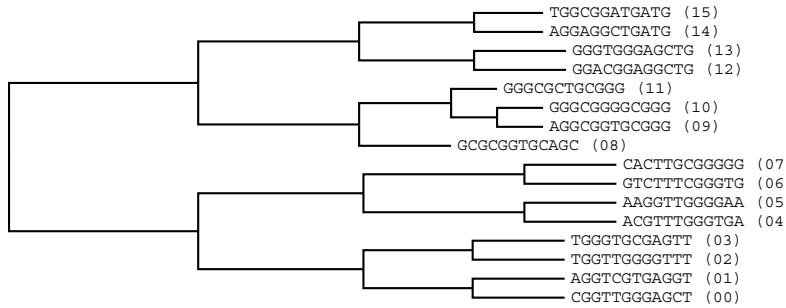


Figure 16.4.4. Example of WPGMA agglomerative clustering on a toy problem. Strings were mutated hierarchically from GGGGGGGGGGGG to produce the input data. WPGMA and related methods (UPGMA, SLC, CLC) yield perfect results for perfectly ultrametric input data, but can deviate badly when that assumption is violated. In this example, however, it does quite well.

scheme of (16.4.6). The different algorithms are distinguished by their prescriptions for computing the distances to new nodes.

The *weighted pair group method using arithmetic averages (WPGMA)* uses this prescription: If a new cluster k is formed from old clusters i and j, then the distance from k to another active cluster p is

$$d_{pk} = d_{kp} = \tfrac{1}{2}(d_{pi} + d_{pj}) \tag{16.4.11}$$

that is, just the average of distances to the two children.

Implementing code, as a class derived from `Phylagglom`, is

phylo.h

```
struct Phylo_wpgma : Phylagglom {
```
Derived class implementing the WPGMA method. Only need to define functions that are virtual in `Phylagglom`.
```
    void premin(MatDoub &d, VecInt &nextp) {}          No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return 0.5*(d[i][k]+d[j][k]);}                 New-node distance is average.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_wpgma(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                           This call actually makes the tree.
};
```

Figure 16.4.4 shows the result of applying the WPGMA method to the toy data at the beginning of this section, using the Hamming distance as the distance metric. You can see that the tree captures almost all of the correct underlying topology, erring only in its pairing of 09 and 10 and thus missing the correct pairings 08-09 and 10-11.

The *unweighted pair group method using arithmetic averages (UPGMA)* uses

$$d_{pk} = d_{kp} = \frac{n_i d_{pi} + n_j d_{pj}}{n_i + n_j} \tag{16.4.12}$$

Although, paradoxically, UPGMA looks "weighted" while WPGMA looks "unweighted," the names derive from the fact that the UPGMA formula is equivalent

to an *un*weighted average of distances to all of a node's descendant leaves. The UPGMA method is the most widely used of the ultrametric heuristic methods.

Implementing code is

phylo.h
```
struct Phylo_upgma : Phylagglom {
```
Derived class implementing the UPGMA method. Only need to define functions that are virtual in `Phylagglom`.
```
    void premin(MatDoub &d, VecInt &nextp) {}            No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return (ni*d[i][k] + nj*d[j][k]) / (ni+nj);}     Distance is weighted.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_upgma(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                             This call actually makes the
};                                                           tree.
```

The *single linkage clustering method* and the *complete linkage clustering method* use, respectively, the minimum and maximum distances to the two children,

$$\begin{aligned} d_{pk} &= d_{kp} = \min(d_{pi}, d_{pj}) \qquad \text{(single linkage)} \\ d_{pk} &= d_{kp} = \max(d_{pi}, d_{pj}) \qquad \text{(complete linkage)} \end{aligned} \tag{16.4.13}$$

Implementing code is

phylo.h
```
struct Phylo_slc : Phylagglom {
```
Derived class implementing the single linkage clustering method.
```
    void premin(MatDoub &d, VecInt &nextp) {}      No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return MIN(d[i][k],d[j][k]);}          New-node distance is min of children.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_slc(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                       This call actually makes the tree.
};

struct Phylo_clc : Phylagglom {
```
Derived class implementing the complete linkage clustering method.
```
    void premin(MatDoub &d, VecInt &nextp) {}        No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return MAX(d[i][k],d[j][k]);}          New-node distance is max of children.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_clc(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                       This call actually makes the tree.
};
```

16.4.5 Neighbor Joining: Exact for Additive Trees

Saitou and Nei's *neighbor-joining method (NJ)* [6] is an agglomerative method with the remarkable property that it exactly reconstructs any additive tree, given that tree's distance matrix (again modulo some technical details). NJ is probably the most widely used agglomerative method, and perhaps the most widely used method

for phylogenetic tree construction overall. Real biological trees are almost never close enough to ultrametric to give UPGMA a significant advantage over NJ, so NJ is likely the method, among the fast heuristic methods, that you will want to try first.

The prescriptions for treating NJ within the framework of `Phyloagglom` are slightly more complicated than for the ultrametric heuristics. At each stage of forming a new cluster, we compute an auxiliary quantity,

$$u_i \equiv \frac{1}{n_a - 2} \sum_{j \neq i} d_{ij} \tag{16.4.14}$$

where the sum is over active clusters, whose number is n_a. Then, we find not the minimum distance, per se, but the minimum of the expression

$$d_{ij} - u_i - u_j \tag{16.4.15}$$

When we connect clusters i and j to form a new node k, the branch lengths from i to k and from j to k are

$$\begin{aligned} d_{ik} &= \tfrac{1}{2}(d_{ij} + u_i - u_j) \\ d_{jk} &= \tfrac{1}{2}(d_{ij} + u_j - u_i) \end{aligned} \tag{16.4.16}$$

Finally, the distance between new node k and another node p is

$$d_{pk} = d_{kp} = \tfrac{1}{2}(d_{pi} + d_{pj} - d_{ij}) \tag{16.4.17}$$

(You can now see why `Phylagglom` was coded with some features that were not exercised by the ultrametric heuristics.)

phylo.h

```
struct Phylo_nj : Phylagglom {
Derived class implementing the neighbor joining (NJ) method.
    VecDoub u;
    void premin(MatDoub &d, VecInt &nextp) {
    Before finding the minimum we (re-)calculate the u's.
        Int i,j,ncurr = 0;
        Doub sum;
        for (i=0; i>=0; i=nextp[i]) ncurr++;          Count live entries.
        for (i=0; i>=0; i=nextp[i]) {                 Compute u[i].
            sum = 0.;
            for (j=0; j>=0; j=nextp[j]) if (i != j) sum += d[i][j];
            u[i] = sum/(ncurr-2);
        }
    }
    Doub dminfn(MatDoub &d, Int i, Int j) {
        return d[i][j] - u[i] - u[j];                 NJ finds min of this.
    }
    Doub dbranchfn(MatDoub &d, Int i, Int j) {
        return 0.5*(d[i][j]+u[i]-u[j]);               NJ setting for branch lengths.
    }
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return 0.5*(d[i][k] + d[j][k] - d[i][j]);     NJ new distances.
    }
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
    Doub &bi, Doub &bj) {
        Since NJ is unrooted, it is a matter of taste how to assign branch lengths to the root.
        This prescription plots aesthetically.
        bi = d[i][j]*(nj - 1 + 1.e-15)/(ni + nj -2 + 2.e-15);
```

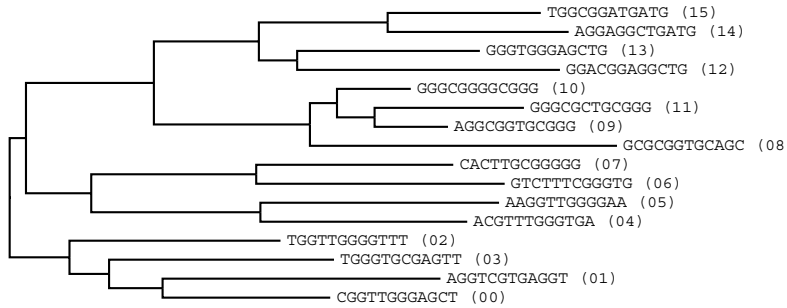


Figure 16.4.5. Same data as previous figure, clustered by the neighbor-joining (NJ) method. The method yields perfect results when the input data are the metric of an additive tree (which these data are not). While displayed here as if rooted, the NJ method outputs an unrooted tree (see next figure).

```
        bj = d[i][j]*(ni - 1 + 1.e-15)/(ni + nj -2 + 2.e-15);
    }
    Phylo_nj(const MatDoub &dist, Int fsr = -1)
        : Phylagglom(dist,fsr), u(n) {makethetree(dist);}
};
```

Computing the u_i's is here coded as an $O(n^2)$ process, but it is repeated $O(n)$ times, so it adds $O(n^3)$ to the workload. It is straightforward to make this $O(n^2)$, in line with the best coding for the rest of the tree construction. When you compute the u_i's, most distances have not changed; you just need to correct those that have. We have not coded this, just to keep the code concise.

It is important to keep in mind that the neighbor-joining method intrinsically produces an *unrooted* tree, regardless of how the graphical output may be drawn. Figure 16.4.5 shows the tree produced by the above code, run on the same toy example as above. It is clear by inspection that, if we want to root the tree at all, we will likely do so at some different point than the one drawn. It is for just this reason that `Phylo_nj`'s constructor has an optional integer argument for specifying a node as an immediate daughter to a "forced" root. (You can't specify a new root by its node number, because it doesn't exist yet.) Also, since you may not know how `Phyloagglom` has numbered its internal nodes, there is a method that returns the node number of an internal node, given two leaves that have it as their first common ancestor.

phylo.h

```
Int Phylagglom::comancestor(Int leafa, Int leafb) {
```

Given the node numbers of two leaves, return the node number of their first common ancestor.

```
    Int i, j;
    for (i = leafa; i != root; i = t[i].mo) {
        for (j = leafb; j != root; j = t[j].mo) if (i == j) break;
        if (i == j) break;
    }
    return i;
}
```

Figure 16.4.6 shows the result of rerooting the tree of Figure 16.4.5 to the common ancestor of leaves 08 and 15. The recovered topology is now seen to be almost identical to that recovered by WPGMA, except for one additional mistake in not giving 02 and 03 a common parent.

The two figures, 16.4.5 and 16.4.6, were produced by lines of code like

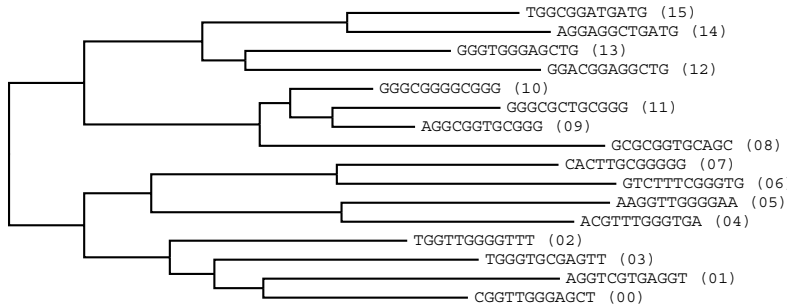


Figure 16.4.6. Same (NJ) tree as previous figure, but displayed with a different root, so as to produce a more balanced tree.

```
Mat_DP dist(n,n);
...
Phylo_nj mytree(dist);
Int i = mytree.comancestor(8,15);
Phylo_nj myrerootedtree(dist,i);
```

Although an unlikely application, you can use neighbor joining to test whether a given distance matrix is additive: Construct the NJ tree, and then see if its induced (branch length) distance matrix is the same as the one you started with. Analogously, you can use any of the ultrametric heuristic methods to test whether a distance matrix is ultrametric.

To view a tree produced by `Phyagglom`, you can use the following routine to produce an output file in the so-called "Newick" or "New Hampshire" format. Most tree viewing programs (e.g., TreeView) can read such a file. Alternatively, see Webnote [10] for a routine to convert a `Phyagglom` to PostScript graphics directly.

```
void newick(Phylagglom &p, MatChar str, char *filename) {                    phylo.h
```
Output a phylogenetic tree p in the "Newick" or "New Hampshire" standard format. Text labels for the leaf nodes are input as the rows of `str`, each a null terminated string. The output file name is specified by `filename`.

```
    FILE *OUT = fopen(filename,"wb");
    Int i, s, ntask = 0, n = p.n, root = p.root;
    VecInt tasklist(2*n+1);
    tasklist[ntask++] = (1 << 16) + root;
    while (ntask-- > 0) {                          Depth-first traversal of the tree.
        s = tasklist[ntask] >> 16;                 Code indicating context.
        i = tasklist[ntask] & 0xffff;              Node number to be processed.
        if (s == 1 || s == 2) {                    Left or right dau, down.
            tasklist[ntask++] = ((s+2) << 16) + p.t[i].mo;
            if (p.t[i].ldau >= 0) {
                fprintf(OUT,"(");
                tasklist[ntask++] = (2 << 16) + p.t[i].rdau;
                tasklist[ntask++] = (1 << 16) + p.t[i].ldau;
            }
            else fprintf(OUT,"%s:%f",&str[i][0],p.t[i].modist);
        }
        else if (s == 3) {if (ntask > 0) fprintf(OUT,",\n");}   Left dau up.
        else if (s == 4) {                                      Right dau up.
            if (i == root) fprintf(OUT,");\n");
            else fprintf(OUT,"):%f",p.t[i].modist);
        }
    }
    fclose(OUT);
}
```

16.4.6 Maximum Likelihood and Maximum Parsimony

Both methods that we now discuss are character-based. If you have a problem that starts with a distance matrix, you can skip this section.

Maximum likelihood (or its Bayesian equivalent, maximum posterior probability) *sounds* like a good idea for phylogenetic inference, but it has two crippling disabilities:

First, its exact solution requires looping over (more or less) all possible trees, so it must confront a super-exponential increase of workload with n, the number of data points.

Second, since you need to compute the probability of each tree, given the terminal node (leaf) data, you need to have a precise statistical model of how trees are generated, i.e., how evolution works. While there are various such models, with varying degrees of support by empirical data, no such model can convincingly claim to be a universal truth. Under different models, maximum likelihood will produce different trees.

There are heuristic methods, e.g. *quartet puzzling* [11], that finesse, to some extent, the first disability, at the price of yielding "solutions" that are not necessasrily the absolute global optimum. However, the combination of both disabilities generally makes maximum likelihood a method of last resort.

Maximum parsimony shares maximum likelihood's first disability, but not its second. Since in many situations it has proved itself to be an accurate and robust method [5], a lot of work has been done on heuristics that can overcome its workload limitations, again at the price of yielding only approximate solutions, and with significant success.

The basic idea of maximum parsimony is very simple: Consider all trees that have the observed data as their leaves. By "all trees" we mean not just all tree topologies, but rather all actual trees with interior nodes that are fully labeled by posited ancestor sequences. Now, for each such tree, define its branch lengths to be the Hamming distances between parent and child. For example, if a child's sequence differs from its parent's in two character positions, then their connecting branch has length two. The parsimony score for a tree is the sum of all of its branch lengths. The maximum parsimony tree is the tree with the smallest parsimony score.

It turns out that one subpart of this search can be done in a computationally efficient way. The *small parsimony problem* is: Given the *topology* of a tree over the observed leaves, find the maximum parsimony way to assign sequences to all the internal nodes. *Fitch's algorithm*, which is beyond our scope to describe, solves this in $O(n\,m\,k)$ time, where m is the length of the sequences and k is the number of possible values for each character (e.g., $k = 4$ for DNA bases) [1].

The hard part of maximum parsimony is therefore the search over topologies. When n is less than around 17, exhaustive search is possible. For larger n, various techniques including random addition order, branch swapping, hill climbing, branch and bound, simulated annealing, and genetic algorithms are used singly and in combination. In general, these can give a result with only a local maximum parsimony; but the results are often very good [1,5]. Unfortunately, the details are all beyond our allowed scope here. PAUP [7] and TNT [12,13] both implement sophisticated maximum parsimony searches.

CITED REFERENCES AND FURTHER READING:

Felsenstein, J. 2004, *Inferring Phylogenies* (Sunderland, MA: Sinauer).[1]

Warnow, T. 1996, "Some Combinatorial Optimization Problems in Phylogenetics," in *Proceedings of the International Colloquium on Combinatorics and Graph Theory*, eds. A. Gyarfas, L. Lovasz, L.A. Szekely, Bolyai Society Mathematical Studies, vol. 7, (Budapest: Bolyai Society).[2]

Agarwala, R. et al. 1999, "On the Approximability of Numerical Taxonomy," *SIAM Journal of Computing*, vol. 28, no. 3, pp. 1073-1085.[3]

Cohen, J. and Farach M. 1997, in *Proceedings of the 8th Annual ACM-SIAM Symposium on Discrete Algorithms SODA '97* (Philadelphia: S.I.A.M.).[4]

Rice, K. and Warnow, T. 1997, "Parsimony Is Hard to Beat," in *Computing and Combinatorics*, 3rd Annual International Conference COCOON '97, T. Jiang and D.T. Lee, eds. (New York: Springer).[5]

Saitou, N. and Nei, M. 1987, "The Neighbor-joining Method: A New Method for Reconstructing Phylogenetic Trees," *Molecular Biology and Evolution*, vol. 4, no. 4, pp. 406-425; see also *op. cit.*, vol. 5, no. 6, pp. 729-731.[6]

Swofford, D. 2004+, *PAUP: Phylogenetic Analysis Using Parsimony and Other Methods*, version 4, at `http://paup.csit.fsu.edu/`.[7]

Felsenstein, J. 2003+, *PHYLIP: Phylogeny Inference Package*, version 3.6, at `http://evolution.genetics.washington.edu/phylip.html`.[8]

Hall, B.G. 2004, *Phylogenetic Trees Made Easy: A How-To Manual*, 2nd ed. (Sunderland, MA: Sinauer).[9]

Numerical Recipes Software 2007, "Code for Rendering a Phylagglom Tree in Simple PostScript," *Numerical Recipes Webnote No. 28*, at `http://www.nr.com/webnotes?28` [10]

Strimmer, K. and von Haeseler, A. 1996, "Quartet Puzzling: A Quartet Maximum Likelihood Method for Reconstructing Tree Topologies," *Molecular Biology and Evolution*, vol. 13, no. 7, pp. 964-969.[11]

Goloboff, P.A. 1999, "Analyzing Large Data Sets in Reasonable Times: Solutions for Composite Optima," *Cladistics*, vol. 15, pp. 415-428; also see `http://www.cladistics.com`.[12]

Nixon, K.C. 1999, "The Parsimony Ratchet, a New Method for Rapid Parsimony Analysis," *Cladistics*, vol. 15, pp. 407-414.[13]

16.5 Support Vector Machines

The *support vector machine* or *SVM*, first described by Vapnik and collaborators in 1992 [1], has rapidly established itself as a powerful algorithmic approach to the problem of *classification* within the larger context known as *supervised learning*. SVMs are no more "machines" than are Turing "machines"; the use of the word is inherited from that part of computer science long known as "machine learning." A number of classification problems whose solutions were previously dominated by neural nets and more complicated methods have been found to be straightforwardly solvable by SVMs [2]. Moreover, SVMs are generally easier to implement than are neural nets; and it is generally easier to intuit what SVMs "think they are doing" than for neural nets, which are famous for their opaqueness.

In the supervised learning problem of classification, we are given a set of *training data* consisting of m points,

$$(\mathbf{x}_i, y_i) \qquad i = 1, \ldots, m \tag{16.5.1}$$

Each $\mathbf{x}_i$ is a *feature vector* in n dimensions (say) that describes the data point, while each corresponding y_i has the value ± 1, indicating whether that data point is in $(+1)$ or out of (-1) the set that we want to learn to recognize. We desire a *decision rule*, in the form of a function $f(\mathbf{x})$ whose sign predicts the value of y, not just for the data in the training set, but also for new values of $\mathbf{x}$ never before seen.

For some applications, the feature vector $\mathbf{x}$ truly lives in the continuous space $\mathbf{R}^n$. However, you are allowed to be creative in mapping your problem into this framework: In many applications, the feature vector will be a binary vector that encodes the presence or absence of many "features" (hence its name). For example, the feature vector describing a DNA sequence of length p could have $n = 4p$ dimensions, with each base position using four dimensions, and having the value one in one of the four (depending on whether it is A, C, G, or T), zero in the others.

16.5.1 Special Case of Linearly Separable Data

One can understand SVMs conceptually as a series of generalizations from an idealized, and rather unrealistic, starting point, We discuss these generalizations sequentially in the rest of this section. The starting point is the special case of *linearly separable data*. In this case, we are told (by an oracle?) that there exists a hyperplane in n dimensions, that is, an $n-1$ dimensional surface defined by the equation

$$f(\mathbf{x}) \equiv \mathbf{w} \cdot \mathbf{x} + b = 0 \tag{16.5.2}$$

that completely separates the training data. In other words, all the training points with $y_i = 1$ lie on one side of the hyperplane (and thus have $f(\mathbf{x}_i) > 0$), while all the training points with $y_i = -1$ lie on the other side (and have $f(\mathbf{x}_i) < 0$). All we have to do is find $\mathbf{w}$ (a normal vector to the hyperplane) and b (an offset). Then $f(\mathbf{x})$ in equation (16.5.2) will be the decision rule.

Actually, we can do better than this. In general, more than one hyperplane will separate linearly separable data. Let's pick the hyperplane that has the largest *margin*, i.e., maximizes the perpendicular distance to points nearest to the hyperplane on both sides. Specifically, given any hyperplane that separates the data, we can always scale $\mathbf{w}$ by a constant and adjust b appropriately, to make

$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b &\geq +1 \quad \text{when } y_i = +1 \\ \mathbf{w} \cdot \mathbf{x}_i + b &\leq -1 \quad \text{when } y_i = -1 \end{aligned} \tag{16.5.3}$$

These equations represent parallel bounding hyperplanes that separate the data (see Figure 16.5.1), a structure whimsically called a *fat plane*. With a bit of analytical geometry, you can easily convince yourself that the perpendicular distance between the bounding hyperplanes (twice the margin) is

$$2 \times \text{margin} = 2(\mathbf{w} \cdot \mathbf{w})^{-1/2} \tag{16.5.4}$$

Also note that both cases of equation (16.5.3) can be summarized as the single equation

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \tag{16.5.5}$$

What we see is that the fattest fat plane, also known as the *maximum margin SVM*, can be found by solving a particular problem in quadratic programming:

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{w} \cdot \mathbf{w} \\ \text{subject to:} & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \qquad i = 1, \ldots, m \end{array} \tag{16.5.6}$$

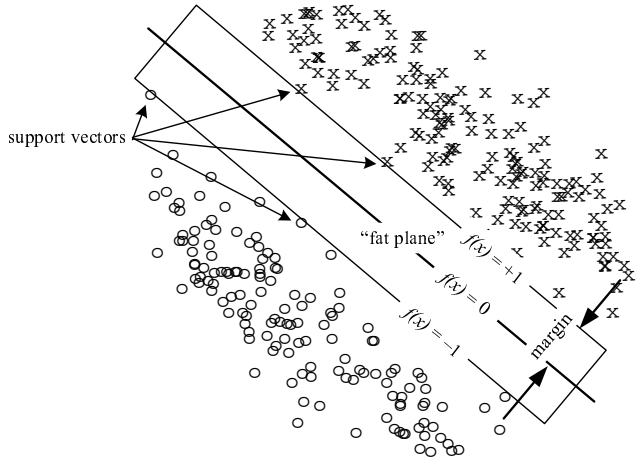


Figure 16.5.1. Support vector machine (SVM) in the idealized case of linearly separable data. We want to classify regions of the plane as containing x's or o's. The "fat plane" defined by $-1 \le f(x) \le 1$ is chosen to maximize the margin (shown). At such a maximum, a small number of points, the "support vectors," will lie on the bounding planes.

Note that we minimize $\mathbf{w} \cdot \mathbf{w}$ instead of equivalently maximizing its reciprocal. The factor of $1/2$ merely simplifies some algebra, later.

General methods for solving quadratic programming problems like the above are discussed in [3,4]. Later in this section, we discuss a method specialized for some SVMs. For now, consider the solution of (16.5.6) and similar problems as an available "black box."

At a solution of (16.5.6), some (usually a small number) of the data points must lie exactly on one or the other bounding hyperplane, because, otherwise, the fat plane could have been made fatter. These data points, with $f(\mathbf{x}) = \pm 1$, are called the *support vectors* of the solution. However, despite the fact that support vector machines were originally named after these support vectors, they don't play much of a role in the more realistic generalizations that we will soon discuss.

16.5.2 Primal and Dual Problems in Quadratic Programming

The first of our promised generalizations may at first sight seem a puzzling direction to go, since it consists merely of replacing one quadratic programming problem with another. We will see later, however, that this replacement has profound consequences.

The general problem in quadratic programming, known as the *primal* problem, can be stated as

$$\begin{array}{ll} \text{minimize:} & f(\mathbf{w}) \\ \text{subject to:} & g_j(\mathbf{w}) \le 0 \\ & h_k(\mathbf{w}) = 0 \end{array} \tag{16.5.7}$$

where $f(\mathbf{w})$ is quadratic in $\mathbf{w}$; $g(\mathbf{w})$ and $h(\mathbf{w})$ are affine in $\mathbf{w}$ (i.e., linear plus a constant); and j and k index, respectively, the sets of inequality and equality constraints.

Every primal problem has a *dual* problem, which can be thought of as an alternative way of solving the primal problem (cf. §10.11.1). To get from the primal to the dual, one writes a Lagrangian that incorporates both the quadratic form, and — with Lagrange multipliers — all the constraints, namely,

$$\mathcal{L} \equiv \tfrac{1}{2} f(\mathbf{w}) + \sum_j \alpha_j g_j(\mathbf{w}) + \sum_k \beta_k h_k(\mathbf{w}) \tag{16.5.8}$$

One then writes this subset of conditions for an extremum:

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0, \qquad \frac{\partial \mathcal{L}}{\partial \beta_k} = 0, \tag{16.5.9}$$

and uses the resulting equations algebraically to eliminate $\mathbf{w}$ from $\mathcal{L}$, in favor of $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ (where we now write the α_j's and β_k's as vectors). Call the result, the reduced Lagrangian, $\mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta})$. Then the important result, which follows from the so-called *strong duality* and *Kuhn-Tucker* theorems (cf. §10.11.1), is that the solution of the following dual problem is equivalent to the original primal problem:

maximize:	$\mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta})$	
subject to:	$\alpha_j \geq 0$	for all j

(16.5.10)

In fact, this result is more general than quadratic programming and is true, roughly speaking, for any convex $f(\mathbf{w})$. Furthermore, if $\hat{\mathbf{w}}$ is the optimal solution of the primal problem, and $\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}$ are the optimal solutions of the dual problem, we have

$$\begin{aligned} f(\hat{\mathbf{w}}) &= \mathcal{L}(\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}) \\ \hat{\alpha}_j \, g_j(\hat{\mathbf{w}}) &= 0 \qquad \text{for all } j \end{aligned} \tag{16.5.11}$$

The latter condition is called the *Karush-Kuhn-Tucker complementarity condition*. It says that at least one of $\hat{\alpha}_j$ and $g_j(\hat{\mathbf{w}})$ must be zero for each j. (We previously met the linear case in equation 10.11.5.) This means that, from the solution of the dual problem, you can instantly identify inequality constraints in the primal problem that are "pinned" against their limit, namely those with nonzero $\hat{\alpha}_j$'s in the solution of the dual.

16.5.3 Dual Formulation of the Maximum Margin SVM

The above procedure is readily performed on the quadratic programming problem (16.5.6) for the maximum margin SVM. There are no β_k's, since there are no equality constraints. The Lagrangian (16.5.8) is

$$\mathcal{L} = \tfrac{1}{2}\mathbf{w} \cdot \mathbf{w} + \sum_i \alpha_i [1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b)] \tag{16.5.12}$$

The conditions for an extremum are

$$0 = \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i \quad \Longrightarrow \quad \hat{\mathbf{w}} = \sum_i \hat{\alpha}_i y_i \mathbf{x}_i \tag{16.5.13}$$

and

$$0 = \frac{\partial \mathcal{L}}{\partial b} = \sum_i \alpha_i y_i \tag{16.5.14}$$

Substituting equations (16.5.13) and (16.5.14) back into (16.5.12) gives the reduced Lagrangian

$$\begin{aligned} \mathcal{L}(\boldsymbol{\alpha}) &= \sum_j \alpha_j - \tfrac{1}{2} \sum_{j,k} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_k) y_k \alpha_k \\ &\equiv \mathbf{e} \cdot \boldsymbol{\alpha} - \tfrac{1}{2} \boldsymbol{\alpha} \cdot \mathrm{diag}(\mathbf{y}) \cdot \mathbf{G} \cdot \mathrm{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} \end{aligned} \tag{16.5.15}$$

In the second form of the above equation we introduce some convenient matrix notation: $\mathbf{e}$ is the vector whose components are all unity, diag denotes a diagonal matrix formed from a vector in the obvious way, and $\mathbf{G}$ is the *Gram matrix* of dot products of all the $\mathbf{x}_j$'s,

$$G_{ij} \equiv \mathbf{x}_i \cdot \mathbf{x}_j \tag{16.5.16}$$

Remember that subscripts on $\mathbf{x}$ don't indicate components, but rather index which data point is referenced.

The dual problem, in toto, thus turns out to be

minimize:	$\frac{1}{2} \boldsymbol{\alpha} \cdot \mathrm{diag}(\mathbf{y}) \cdot \mathbf{G} \cdot \mathrm{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} - \mathbf{e} \cdot \boldsymbol{\alpha}$	
subject to:	$\alpha_i > 0$	for all i
	$\boldsymbol{\alpha} \cdot \mathbf{y} = 0$	(from 16.5.14)

(16.5.17)

We also have the Karush-Kuhn-Tucker relation,

$$\hat{\alpha}_i \, [y_i (\hat{\mathbf{w}} \cdot \mathbf{x}_i + b) - 1] = 0 \tag{16.5.18}$$

Equation (16.5.13) tells how to get the optimal solution $\hat{\mathbf{w}}$ of the primal problem from the solution $\hat{\boldsymbol{\alpha}}$ of the dual. Equation (16.5.18) is then used to get $\hat{b}$: Find *any* nonzero α_i, then, with the corresponding y_i, $\mathbf{x}_i$, and $\hat{\mathbf{w}}$, solve the above relation for $\hat{b}$. Alternatively, one can average out some roundoff error by taking a weighted average of α_i's,

$$\hat{b} = \sum_i \hat{\alpha}_i (y_i - \hat{\mathbf{w}} \cdot \mathbf{x}_i) \Big/ \sum_i \alpha_i \tag{16.5.19}$$

Finally, the decision rule is $f(\mathbf{x}) = \hat{\mathbf{w}} \cdot \mathbf{x} + \hat{b}$.

A few observations will become important later:

- Data points with nonzero $\hat{\alpha}_i$ satisfy the constraints as equalities, i.e., they are support vectors.
- The only place that the data $\mathbf{x}_i$'s appear in (16.5.17) is in the Gram matrix $\mathbf{G}$.
- The only part of the calculation that scales with n (the dimensionality of the feature vector) is computing the components of the Gram matrix.
- All other parts of the calculation scale with m, the number of data points.

Thus, in going from primal to dual, we have substituted for a problem that scales (mostly) with the dimensionality of the feature matrix a problem that scales (mostly) with the number of data points. This might seem odd, because it makes problems with huge numbers of data points difficult. However, it makes easy, as we will soon see, problems with moderate amounts of data but *huge* feature vectors. This is in fact the regime where SVMs really shine.

16.5.4 The 1-Norm Soft-Margin SVM and Its Dual

The next important generalization is to relax the unrealistic assumption that there exists a hyperplane that separates the training data, i.e., get rid of the "oracle." We do this by introducing a so-called slack variable ξ_i for each data point $\mathbf{x}_i$. If the data point is one that *can* be separated by a fat plane, then $\xi_i = 0$. If it *can't* be, then $\xi_i > 0$ is the amount of the discrepancy, expressed by the modified inequality

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \tag{16.5.20}$$

We must of course build in an inducement for the optimization to make the ξ_i's as small as possible, zero whenever possible. We thus have a trade-off between making the ξ_i's small and making the fat plane fat. In other words, we now have a problem that requires not only optimization, but also *regularization*, in the same sense as the discussion in §18.4. In the notation of equation (18.4.12), our quadratic forms ($\mathbf{w} \cdot \mathbf{w}$ or $\mathcal{L}$) are examples of $\mathcal{A}$'s. We need to invent a regularizing operator $\mathcal{B}$ that expresses our hopes for the ξ_i's, and then minimize $\mathcal{A} + \lambda\mathcal{B}$, instead of just $\mathcal{A}$ alone. As we vary λ in the range $0 < \lambda < \infty$, we explore a regularization trade-off curve.

The *1-norm soft-margin SVM* adopts, as the name indicates, a linear sum of the (positive) ξ_i's as its regularization operator. The primal problem is thus

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{w} \cdot \mathbf{w} + \lambda \sum_i \xi_i \\ \text{subject to:} & \xi_i \geq 0, \\ & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \qquad i = 1, \ldots, m \end{array} \tag{16.5.21}$$

A possible variant is the *2-norm soft-margin SVM*, where the regularization term would be $\sum_i {\xi_i}^2$; however, this gives somewhat more complicated equations, so we will put it beyond our scope here.

Along the trade-off curve $0 < \lambda < \infty$, we vary from a a solution that prefers a *really fat* fat plane (no matter how many points are inside, or on the wrong side, of it) to a solution that is so miserly in allowing discrepancies that it settles for a fat plane with hardly any margin at all. The former is less accurate on the training data but possibly more robust on new data; the latter is as accurate as possible on the training data but possibly fragile (and less accurate) on new data. As in Chapter 19, the choice of λ is a design trade-off that you have to make. (However, we give you some guidance, below.)

Importantly, *any* nonnegative value of λ allows there to be *some* solution, whether the data are linearly separable or not. You can see this by noting that $\mathbf{w} = 0$ is always a feasible (but not optimal) solution of (16.5.21) for sufficiently large positive ξ_i's, no matter what the value of λ. If there is a feasible solution, there must, of course, be an optimal solution.

The very astute reader might notice that λ here seems to have the opposite qualitative sense from the λ's in Chapter 19. Specifically, $\lambda \to 0$ (here) gives the "softer," more robust, solution, while in Chapter 19 it is $\lambda \to \infty$ that, in a similar way, favors a priori smoothness. The reason for this switch is that the quadratic program (16.5.21) becomes the quadratic program (16.5.6) in the limit $\lambda \to \infty$, not 0. This is because there are no ξ_i's in the constraints in (16.5.6), so (16.5.21) must, in

the limit, force them to zero, requiring infinite λ. Correspondingly, as λ approaches zero, the ξ_i's become unconstrained. So the regularization term here indeed does act with the opposite sense from Chapter 19, because of the way it acts through the constraints, not the main functional.

Curiously, the dual to the 1-norm soft-margin SVM turns out to be *almost* identical to the dual of the (unrealistic) maximum margin SVM (16.5.17). Omitting details of the calculation, the result is

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\boldsymbol{\alpha}\cdot\text{diag}(\mathbf{y})\cdot\mathbf{G}\cdot\text{diag}(\mathbf{y})\cdot\boldsymbol{\alpha}-\mathbf{e}\cdot\boldsymbol{\alpha} \\ \text{subject to:} & 0\le\alpha_i\le\lambda \qquad \text{for all } i \\ & \alpha\cdot\mathbf{y}=0 \end{array} \tag{16.5.22}$$

That is, the only difference is that there is now a constraining *upper* bound of λ on α_i in addition to the *lower* bound of zero. (This kind of constraint is called a *box constraint*.)

The formula for $\hat{\mathbf{w}}$ is unchanged from equation (16.5.13), while the Karush-Kuhn-Tucker conditions now become

$$\begin{aligned} (\hat{\alpha}_i-\lambda)\hat{\xi}_i &= 0 \\ \hat{\alpha}_i\left[y_i(\hat{\mathbf{w}}\cdot\mathbf{x}_i+\hat{b})-1+\hat{\xi}_i\right] &= 0 \end{aligned} \tag{16.5.23}$$

We see that, except for rare degenerate cases of double zeros,

$$\begin{array}{rcl} \hat{\alpha}_i=0 & \Longleftrightarrow & \text{data point } i \text{ on correct side of fat plane} \\ 0<\hat{\alpha}_i<\lambda & \Longleftrightarrow & \text{data point } i \text{ exactly on fat plane boundary (a support vector)} \\ \hat{\alpha}_i=\lambda & \Longleftrightarrow & \text{data point } i \text{ inside, or on wrong side, of fat plane} \end{array} \tag{16.5.24}$$

Here again we see that, as we reduce λ toward zero, pinning more and more α_i's at the value λ, we get solutions with increasing numbers of "wrong" points, but fatter fat planes.

The roundoff-averaged estimator for $\hat{b}$, analogous to equation (16.5.19), is

$$\hat{b}=\sum_i\hat{\alpha}_i(\lambda-\hat{\alpha}_i)(y_i-\hat{\mathbf{w}}\cdot\mathbf{x}_i)\Big/\sum_i\hat{\alpha}_i(\lambda-\hat{\alpha}_i) \tag{16.5.25}$$

Although the linear assumption (that is, using hyperplanes to separate the data) is still somewhat restrictive, the model defined by (16.5.22) does have some practical utility in problems where there is some reason to believe that the response is (at least somewhat) linear in the components of the feature vector. But that is far from the end of the story.

16.5.5 The Kernel Trick

Finally, we get to the generalization that gives SVMs their real power. Imagine an embedding function $\boldsymbol{\varphi}$ that maps n-dimensional feature vectors, in some manner, into a much higher N-dimensional space,

$$\mathbf{x}\quad(n\text{-dimensional})\quad\longrightarrow\quad\boldsymbol{\varphi}(\mathbf{x})\quad(N\text{-dimensional}) \tag{16.5.26}$$

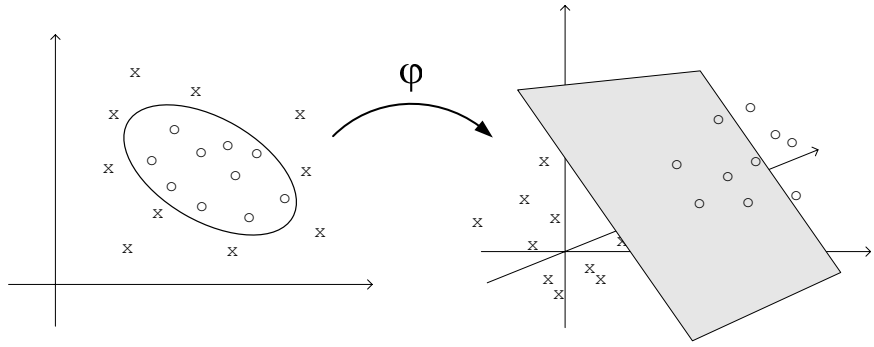


Figure 16.5.2. When feature vectors are mapped from a lower-dimensional space (here 2) to a higher-dimensional embedding space (here 3), nonlinear separation surfaces can become well approximated by linear ones. In practice, *very* high-dimensional embedding spaces are used, but they enter the SVM calculation only implicitly, through the "kernel trick."

The basic idea, as shown in Figure 16.5.2, is that a very nonlinear separating surface in the n-dimensional space might map into (or be well approximated by) a linear hyperplane in the N-dimensional space.

To see why this might work, consider this mapping from two to five dimensions:

$$(x_0, x_1) \quad \xrightarrow{\varphi} \quad (x_0^2, x_0 x_1, x_1^2, x_0, x_1) \tag{16.5.27}$$

With this mapping, a decision rule $f(\mathbf{x})$ that is constructed as linear in the embedding space becomes general enough to include all linear and quadratic forms (lines, ellipses, hyperbolas) in the original feature space, namely,

$$f(\mathbf{x}) = F[\boldsymbol{\varphi}(\mathbf{x})] \equiv \mathbf{W} \cdot \boldsymbol{\varphi}(\mathbf{x}) + B \tag{16.5.28}$$

where we are using uppercase letters for quantities in the embedding space. Although $N = 5$ in this example, it might instead have a value like a million or a billion (we'll see how this works in a minute).

Give our data, how do we find $\mathbf{W}$ and B in the embedding space? Let's try exactly as before, but just in the higher-dimensional space. The primal problem (compare to equation 16.5.21) is

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{W} \cdot \mathbf{W} + \lambda \sum_i \Xi_i \\ \text{subject to:} & \Xi_i \geq 0, \\ & y_i(\mathbf{W} \cdot \boldsymbol{\varphi}(\mathbf{x}_i) + B) \geq 1 - \Xi_i \qquad i = 1, \ldots, m \end{array} \tag{16.5.29}$$

Uh-oh! This is a quadratic programming problem in a million- or billion-dimensional space, not likely to be tractable on your ordinary desktop computer.

What about the dual problem? It turns out to be

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\boldsymbol{\alpha} \cdot \text{diag}(\mathbf{y}) \cdot \mathbf{K} \cdot \text{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} - \mathbf{e} \cdot \boldsymbol{\alpha} \\ \text{subject to:} & 0 \leq \alpha_i \leq \lambda \qquad \text{for all } i \\ & \alpha \cdot \mathbf{y} = 0 \end{array} \tag{16.5.30}$$

This is exactly the same as (16.5.22), except that the Gram matrix G_{ij} has been replaced by the so-called *kernel* K_{ij},

$$K_{ij} \equiv K(\mathbf{x}_i, \mathbf{x}_j) \equiv \boldsymbol{\varphi}(\mathbf{x}_i) \cdot \boldsymbol{\varphi}(\mathbf{x}_j) \tag{16.5.31}$$

Well, this is progress. The quadratic programming problem (16.5.30) is no harder than the original problem (16.5.22)! Both live in a space of dimension m, the number of data points, and both get fed a fixed matrix, precalculated from the data: G_{ij} in one case, K_{ij} in the other.

We have succeeded in maneuvering the "curse of dimensionality" into a very tight corner, namely the calculation of just the m^2 values K_{ij}. Now we annihilate it entirely with *the kernel trick*:

The "trick" is that we never really need to know the mapping $\boldsymbol{\varphi}(\mathbf{x})$ at all. All we need is a way of computing a kernel K_{ij} that *could* have come from some mapping $\boldsymbol{\varphi}(\mathbf{x})$, that is, a matrix of size $m \times m$ with the mathematical properties of an inner product space in higher dimension. We already know one possible kernel, namely the Gram matrix G_{ij}. Here are some provable properties of kernel functions $K(\mathbf{x}_i, \mathbf{x}_j)$ in general:

- $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ must be symmetric in i and j and must have nonnegative eigenvalues (Mercer's theorem).
- Any multinomial combination of kernel functions is a kernel function. That is, you can freely combine kernel functions by multiplication, addition, and scaling by a constant.
- $K(\boldsymbol{\varphi}(\mathbf{x}_i), \boldsymbol{\varphi}(\mathbf{x}_j))$ is a kernel if $K(,)$ is one, for any $\boldsymbol{\varphi}$. This generalizes the original idea of the embedding space.
- $K(\mathbf{x}_i, \mathbf{x}_j) = g(\mathbf{x}_i) g(\mathbf{x}_j)$ is always a kernel, for any function g.

Once you settle on a kernel and solve the quadratic programming problem (16.5.30), then your final decision rule for any new feature vector $\mathbf{x}$ is

$$f(\mathbf{x}) = \sum_i \hat{\alpha}_i y_i K(\mathbf{x}_i, \mathbf{x}) + \hat{b} \tag{16.5.32}$$

where (again using the averaging trick)

$$\hat{b} = \sum_i \hat{\alpha}_i (\lambda - \hat{\alpha}_i)[y_i - \sum_j \hat{\alpha}_i y_j K(\mathbf{x}_j, \mathbf{x}_i)] \Big/ \sum_i \hat{\alpha}_i (\lambda - \hat{\alpha}_i) \tag{16.5.33}$$

While the construction of the ideal kernel for any particular problem can involve some art, some very generic kernels turn out to be quite powerful in solving real-world problems. Often you can just try a few of these and pick the one that works best. The following are good ones to try first:

$$\begin{aligned}
\text{linear:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j \\
\text{power:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j)^d, \qquad 2 \le d \le 20 \text{ (say)} \\
\text{polynomial:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = (a\, \mathbf{x}_i \cdot \mathbf{x}_j + b)^d \\
\text{sigmoid:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(a\, \mathbf{x}_i \cdot \mathbf{x}_j + b) \\
\text{Gaussian radial basis function:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\tfrac{1}{2}|\mathbf{x}_i - \mathbf{x}_j|^2/\sigma^2)
\end{aligned} \tag{16.5.34}$$

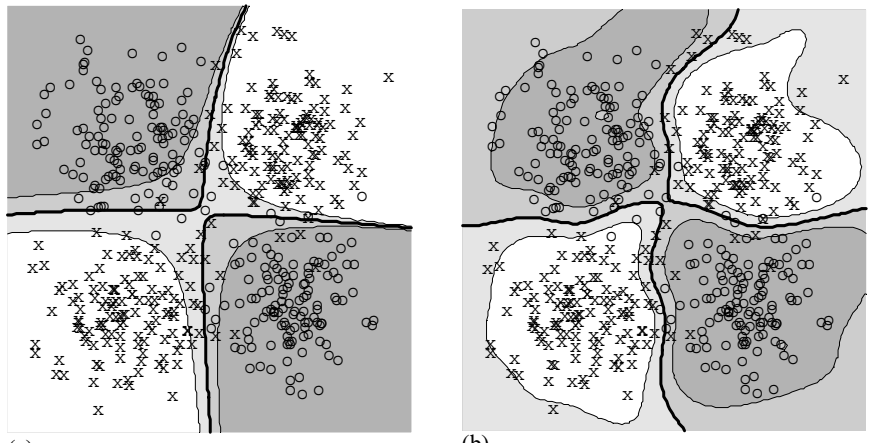


Figure 16.5.3. SVMs that "learn" to partition the plane. The input data are drawn from four two-dimensional Gaussians, slightly overlapping, with diagonally opposite ones given the same label (x or o). Heavy solid lines are the decision rule surfaces $f(\mathbf{x}) = 0$ derived by the SVMs. Lighter lines show $f(\mathbf{x}) = \pm 1$. (a) Polynomial kernel with $d = 8$. (b) Gaussian radial basis function kernel.

See §2.3 of [5] for additional standard kernels. Chapter 13 of [5] describes many specialized kernels, e.g., for comparing strings or passages of text, for image recognition, and for a number of other applications.

Figure 16.5.3 shows a test example using both a polynomial kernel with $d = 8$ and a Gaussian radial basis kernel. It is characteristic of the Gaussian kernel that it is more influenced by local nearest neighbor effects (which may be good or bad, depending on the application), while polynomial kernels seek smoother, more global solutions.

Although it is beyond our scope in this section, we should mention that the kernel trick is applicable not only to SVMs (that is, to algorithms based on separating hyperplanes), but also to a number of other pattern recognition algorithms, for example principal component analysis (PCA) and the Fisher discriminant algorithm. See [5] and [6] for extensive treatments of these *kernel-based learning algorithms*.

16.5.6 Some Practical Advice on SVMs

The Gaussian radial basis function kernel is very popular, because it has only one adjustable parameter, σ, and it is easy to guess a first value to try, namely any characteristic distance between nearby points in the feature space. As mentioned, the Gaussian kernel classifies to some degree by the local neighborhood consensus.

For the polynomial kernels, start by choosing a and b to make $a\,\mathbf{x}_i \cdot \mathbf{x}_j + b$ lie between ± 1 for all i and j. The power d has a (very rough) interpretation as how many different features you want the comparison to "mix." That is, $d = 1$ (linear) partitions the space by one feature at at time; $d = 2$ looks at pairs of features simultaneously, and so on. Also very roughly, the difference between power and polynomial is whether you want to consider only exactly d features at a time (power), or all combinations of d or fewer features (polynomial). These interpretations should not be taken too seriously, however. Specifically, larger d is not always better.

We have not said much about how to choose λ, the regularization parameter.

Try $\lambda = 1$ first, then try increasing and decreasing it by factors of 10. There is typically a broad plateau, as a function of $\log_{10}(\lambda)$, where the precise value of λ doesn't matter much. There is some belief that $\langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle^{-1}$ or $\langle K(\mathbf{x}_i, \mathbf{x}_j) \rangle^{-1}$, where angle brackets denote averages over all i, j pairs, are good starting guesses; but for properly scaled kernels these should not be too different from unity in any case.

As you vary λ and repeatedly solve the quadratic program, look at the fraction of α_i's that are pinned at zero, pinned at λ, or floating between these limits. A good profile will often have the biggest fraction at zero, a smaller (but not necessarily much smaller) fraction at λ, and the smallest fraction in between (see equation 16.5.24 for interpretation). These fractions are also often good indicators for adjusting parameters in your kernel. Naturally you will also be looking at the fraction of your training data that is predicted correctly, that is, has $y_i f(\mathbf{x}_i) > 0$.

Below, we will give a short, self-contained program for finding the solution to SVMs; but for anything other than small problems you will want to use a more sophisticated software package. There are many tricks and shortcuts that can speed the solution of an SVM relative to the general problem of quadratic programming — good SVM packages take advantage of these. For example, a good package should take advantage of sparseness in the feature vectors to save on computation. Our favorite package is Thorsten Joachims' *SVMlight* [7], available for free on the Web. *Gist* [8] is another popular free implementation. The Web site cited in [2] has a page with links to a wide variety of SVM software.

16.5.7 The Mangasarian-Musicant Variant and Its Solution by SOR

Mangasarian and Musicant [9,10] have suggested a very slight variant of equation (16.5.21), and its kernel generalization, that has the interesting property that it can be solved, quite compactly, by the method of successive overrelaxation (SOR; see §20.5.1). In particular, a complete SVM solution program using SOR is less than 100 lines long. We discuss this *M-M variant* here, and implement it in code, just because of its brevity. We have used this code for problems of up to several thousand data points, with feature vectors of length several hundred. Such problems take seconds to solve on a desktop machine. For larger problems, our advice is that you use the more efficient specialized packages [7,8]. *SVMlight*, for example, is typically about an order of magnitude faster than the code we give below.

The primal problem in the 1-norm soft-margin form of the M-M variant is

$$\begin{aligned} &\text{minimize:} && \tfrac{1}{2}\mathbf{w}\cdot\mathbf{w} + b^2 + \lambda \sum_i \xi_i \\ &\text{subject to:} && \xi_i \geq 0, \\ & && y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i \qquad i = 1, \ldots, m \end{aligned} \tag{16.5.35}$$

The only difference from (16.5.21) is that a term b^2 has been added to the functional that is minimized. On its face, this should have the effect of slightly favoring hyperplanes closer to the origin, all else being equal, an innocuous (albeit arbitrary) change. The real purpose of the b^2 term, however, is its algebraic effect when we calculate the dual problem:

$$\boxed{\begin{array}{ll} \text{minimize:} & \frac{1}{2}\boldsymbol{\alpha}\cdot\text{diag}(\mathbf{y})\cdot(\mathbf{G}+\mathbf{e}\otimes\mathbf{e})\cdot\text{diag}(\mathbf{y})\cdot\boldsymbol{\alpha}-\mathbf{e}\cdot\boldsymbol{\alpha} \\ \text{subject to:} & 0\le\alpha_i\le\lambda \qquad \text{for all } i \end{array}} \tag{16.5.36}$$

Aside from an extra term $\mathbf{e}\otimes\mathbf{e}$ (the matrix of all ones) now added to the Gram matrix, the main change from (16.5.21) is that the equality constraint is gone! This renders the solution *much* more tractable numerically. The dual problem also now has a simpler expression for $\hat{b}$,

$$\hat{b}=\sum_i \hat{\alpha}_i y_i \tag{16.5.37}$$

(As before, $\hat{\mathbf{w}}$ is computed from 16.5.13.)

When we do the kernel trick, the only change in (16.5.36) is to change G_{ij} to K_{ij}. Equation (16.5.37) still holds, but $\hat{b}$ is actually superfluous since the decision rule can be written directly as

$$f(\mathbf{x})=\sum_i \hat{\alpha}_i y_i[K(\mathbf{x}_i,\mathbf{x})+1] \tag{16.5.38}$$

Mangasarian and Musicant have shown that the solution of the M-M variant SVM is often identical to the solution of the the standard 1-norm soft-margin SVM (albeit with a different value of λ) and is almost never significantly different. What is quite different, however, is that (16.5.36) and its kernel version can be solved by the following, linearly convergent, relaxation procedure:

- Define $\mathbf{M}\equiv\text{diag}(\mathbf{y})\cdot(\mathbf{K}+\mathbf{e}\otimes\mathbf{e})\cdot\text{diag}(\mathbf{y})$.
- Initialize all the α_i's to zero.
- Repeat *ad libitum* the relaxation replacement, for $i=1,2,\ldots,m$,

$$\alpha_i \leftarrow \mathcal{P}\left[\alpha_i-\omega\frac{1}{M_{ii}}\left(\sum_j M_{ij}\alpha_j-1\right)\right] \tag{16.5.39}$$

Here $\mathcal{P}$ is the projection operator that just puts α back into its allowed range. [Note the similarity to the method of projection onto convex sets (POCS) in §19.5.2.]

$$\mathcal{P}=\begin{cases}0, & \alpha<0\\ \alpha, & 0\le\alpha\le\lambda\\ \lambda, & \alpha>\lambda\end{cases} \tag{16.5.40}$$

The constant ω is the overrelaxation parameter, exactly as in §20.5.1. You pick it in the range $0<\omega<2$. In our experience, the convergence rate does not depend sensitively on ω. If you don't have a better idea, take $\omega=1.3$.

Our implementation begins with a virtual class that defines the interface to a kernel function,

svm.h
```
struct Svmgenkernel {
```
Virtual class that defines what a kernel structure needs to provide.

`Int m, kcalls;`	No. of data points; counter for kernel calls.
`MatDoub ker;`	Locally stored kernel matrix.
`VecDoub_I &y;`	Must provide reference to the y_i's.
`MatDoub_I &data;`	Must provide reference to the $\mathbf{x}_i$'s.

```
    Svmgenkernel(VecDoub_I &yy, MatDoub_I &ddata)
        : m(yy.size()),kcalls(0),ker(m,m),y(yy),data(ddata) {}
    Every kernel structure must provide a kernel function that returns the kernel for arbitrary
    feature vectors.
    virtual Doub kernel(const Doub *xi, const Doub *xj) = 0;
    inline Doub kernel(Int i, Doub *xj) {return kernel(&data[i][0],xj);}
    Every kernel structure's constructor must call fill to fill the ker matrix.
    void fill() {
        Int i,j;
        for (i=0;i<m;i++) for (j=0;j<=i;j++) {
            ker[i][j] = ker[j][i] = kernel(&data[i][0],&data[j][0]);
        }
    }
};
```

Basically, a kernel structure is required to provide references to the data (the $\mathbf{x}_i$'s) and the $\mathbf{y}_i$'s, a matrix of kernel values for all pairs of data points, and two forms of the kernel function: one with two arbitrary feature vectors as arguments, and another with one argument a data point and the other an arbitrary feature vector. Here are three examples of kernels, for three of the standard kernels in equation (16.5.34), built on the above Svmgenkernel.

svm.h

```
struct Svmlinkernel : Svmgenkernel {
```
Kernel structure for the linear kernel, the dot product of two feature vectors (with overall means of each component subtracted).
```
    Int n;
    VecDoub mu;
    Svmlinkernel(MatDoub_I &ddata, VecDoub_I &yy)
```
Constructor is called with the $m \times n$ data matrix, and the vector of y_i's, length m.
```
        : Svmgenkernel(yy,ddata), n(data.ncols()), mu(n) {
        Int i,j;
        for (j=0;j<n;j++) mu[j] = 0.;
        for (i=0;i<m;i++) for (j=0;j<n;j++) mu[j] += data[i][j];
        for (j=0;j<n;j++) mu[j] /= m;
        fill();
    }
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += (xi[k]-mu[k])*(xj[k]-mu[k]);
        return dott;
    }
};

struct Svmpolykernel : Svmgenkernel {
```
Kernel structure for the polynomial kernel.
```
    Int n;
    Doub a, b, d;
    Svmpolykernel(MatDoub_I &ddata, VecDoub_I &yy, Doub aa, Doub bb, Doub dd)
```
Constructor is called with the $m \times n$ data matrix, the vector of y_i's, length m, and the constants a, b, and d.
```
        : Svmgenkernel(yy,ddata), n(data.ncols()), a(aa), b(bb), d(dd) {fill();}
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += xi[k]*xj[k];
        return pow(a*dott+b,d);
    }
};

struct Svmgausskernel : Svmgenkernel {
```
Kernel structure for the Gaussian radial basis function kernel.
```
    Int n;
```

```
    Doub sigma;
    Svmgausskernel(MatDoub_I &ddata, VecDoub_I &yy, Doub ssigma)
    Constructor is called with the m × n data matrix, the vector of y_i's, length m, and the
    constant σ.
        : Svmgenkernel(yy,ddata), n(data.ncols()), sigma(ssigma) {fill();}
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += SQR(xi[k]-xj[k]);
        return exp(-0.5*dott/(sigma*sigma));
    }
};
```

The above is all prefatory to the SVM solution structure. You declare an instance of `Svm` with your kernel as the argument. It then makes available three functions: `relax` performs one "group" of relaxation steps and returns the norm of how much change in $\boldsymbol{\alpha}$ has occurred. (We define "group" below.) You call `relax` repeatedly, with λ and ω as arguments, until the returned value is small enough: 10^{-3} or 10^{-4} is usually plenty. Then (and only then) you may repeatedly call either of two forms of `predict`, which returns the decision rule $f(\mathbf{x})$. One form of `predict` returns the prediction for data points, the other for arbitrary new feature vectors. If you want to examine the α_i's, or count how many are pinned at 0 or λ, you can examine the vector `alph`.

svm.h

```
struct Svm {
```

Class for solving SVM problems by the SOR method.

```
    Svmgenkernel &gker;                    Reference bound to user's kernel (and data).
    Int m, fnz, fub, niter;
    VecDoub alph, alphold;                 Vectors of α's before and after a step.
    Ran ran;                               Random number generator.
    Bool alphinit;
    Doub dalph;                            Change in norm of the α's in one step.
    Svm(Svmgenkernel &inker) : gker(inker), m(gker.y.size()),
        alph(m), alphold(m), ran(21), alphinit(false) {}
        Constructor binds the user's kernel and allocates storage.
    Doub relax(Doub lambda, Doub om) {
    Perform one group of relaxation steps: a single step over all the α's, and multiple steps
    over only the interior α's.
        Int iter,j,jj,k,kk;
        Doub sum;                          Index when α's are sorted by value.
        VecDoub pinsum(m);                 Stored sums over noninterior variables.
        if (alphinit == false) {           Start all α's at 0.
            for (j=0; j<m; j++) alph[j] = 0.;
            alphinit = true;
        }
        alphold = alph;                    Save old α's.
        Here begins the relaxation pass over all the α's.
        Indexx x(alph);                    Sort α's, then find first nonzero one.
        for (fnz=0; fnz<m; fnz++) if (alph[x.indx[fnz]] != 0.) break;
        for (j=fnz; j<m-2; j++) {              Randomly permute all the nonzero α's.
            k = j + (ran.int32() % (m-j));
            SWAP(x.indx[j],x.indx[k]);
        }
        for (jj=0; jj<m; jj++) {               Main loop over α's.
            j = x.indx[jj];
            sum = 0.;
            for (kk=fnz; kk<m; kk++) {         Sums start with first nonzero.
                k = x.indx[kk];
                sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
            }
            alph[j] = alph[j] - (om/(gker.ker[j][j]+1.))*(gker.y[j]*sum-1.);
```

```
            alph[j] = MAX(0.,MIN(lambda,alph[j]));          Projection operator.
            if (jj < fnz && alph[j]) SWAP(x.indx[--fnz],x.indx[jj]);
        }                                   (Above) Make an α active if it becomes nonzero.
        Here begins the relaxation passes over the interior α's.
        Indexx y(alph);                                     Sort. Identify interior α's.
        for (fnz=0; fnz<m; fnz++) if (alph[y.indx[fnz]] != 0.) break;
        for (fub=fnz; fub<m; fub++) if (alph[y.indx[fub]] == lambda) break;
        for (j=fnz; j<fub-2; j++) {                         Permute.
            k = j + (ran.int32() % (fub-j));
            SWAP(y.indx[j],y.indx[k]);
        }
        for (jj=fnz; jj<fub; jj++) {                        Compute sums over pinned α's just
            j = y.indx[jj];                                     once.
            sum = 0.;
            for (kk=fub; kk<m; kk++) {
                k = y.indx[kk];
                sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
            }
            pinsum[jj] = sum;
        }
        niter = MAX(Int(0.5*(m+1.0)*(m-fnz+1.0)/(SQR(fub-fnz+1.0))),1);
        Calculate a number of iterations that will take about half as long as the full pass just
        completed.
        for (iter=0; iter<niter; iter++) {                  Main loop over α's.
            for (jj=fnz; jj<fub; jj++) {
                j = y.indx[jj];
                sum = pinsum[jj];
                for (kk=fnz; kk<fub; kk++) {
                    k = y.indx[kk];
                    sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
                }
                alph[j] = alph[j] - (om/(gker.ker[j][j]+1.))*(gker.y[j]*sum-1.);
                alph[j] = MAX(0.,MIN(lambda,alph[j]));
            }
        }
        dalph = 0.;                                         Return change in norm of α vector.
        for (j=0;j<m;j++) dalph += SQR(alph[j]-alphold[j]);
        return sqrt(dalph);
    }
    Doub predict(Int k) {
    Call only after convergence via repeated calls to relax. Returns the decision rule f(x) for
    data point k.
        Doub sum = 0.;
        for (Int j=0; j<m; j++) sum += alph[j]*gker.y[j]*(gker.ker[j][k]+1.0);
        return sum;
    }
    Doub predict(Doub *x) {
    Call only after convergence via repeated calls to relax. Returns the decision rule f(x) for
    an arbitrary feature vector.
        Doub sum = 0.;
        for (Int j=0; j<m; j++) sum += alph[j]*gker.y[j]*(gker.kernel(j,x)+1.0);
        return sum;
    }
};
```

Although the enforced brevity doesn't allow for *too* many optimizing tricks, `Svm` does have a couple that bear mentioning:

First, each call to the `relax` routine performs, as previously mentioned, a group of relaxations. Specifically, it does one full relaxation pass over all the α_i's, and then multiple passes over only the "interior" α_i's, i.e., those that are not pinned at either 0 or λ. These passes are typically *much* faster than the full pass, since most variables

are typically pinned. To realize the gain, sums over pinned variables that don't vary are computed only once at the beginning of these multiple passes. The number of such passes is calculated dynamically so as to take about half as long as the full pass just taken.

Second, before each pass (both the full and interior), the order of the variables is randomized by a permutation generated from a `Ran` object (§7.1). This randomization speeds up the convergence by as much as an order of magnitude.

CITED REFERENCES AND FURTHER READING:

Boser, B.E., Guyon, I.M., and Vapnik, V.N. 1992, in D. Haussler, ed., *Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory* (New York: ACM Press).[1]

Christianini, N. and Shawe-Taylor, J. 2000, *An Introduction to Support Vector Machines* (Cambridge, U.K.: Cambridge University Press); related Web site `http://www.support-vector.net`.[2]

Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. 2006, *Nonlinear Programming: Theory and Algorithms*, 3rd ed. (Hoboken, NJ: Wiley).[3]

den Hertog, D. 1994, *Interior Point Approach to Linear, Quadratic and Convex Programming: Algorithms and Complexity* (Dordrecht: Kluwer).[4]

Schölkopf, B. and Smola, A.J. 2002, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (Cambridge, MA: MIT Press).[5]

Shawe-Taylor, J. and Christianini, N. 2004, *Kernel Methods for Pattern Analysis* (Cambridge, UK: Cambridge University Press).[6]

Vapnik, V. 1998, *Statistical Learning Theory* (New York: Wiley).

Joachims, T. 1999–, *SVMlight, Implementing Support Vector Machines in C*, at `http://svmlight.joachims.org`.[7]

Noble, W.S. and Pavlidis, P. 1999–, *Gist: Software Tools for Support Vector Machine Classification and for Kernel Principal Components Analysis*, at `http://microarray.cpmc.columbia.edu/gist`.[8]

Mangasarian, O.L. and Musicant, D.R. 1999, "Successive Overrelaxation for Support Vector Machines," *IEEE Transactions on Neural Networks*, vol. 10, no. 5, p. 1032.[9]

Mangasarian, O.L. and Musicant, D.R. 2001, in *Complementarity: Applications, Algorithms and Extensions*, M.C. Ferris, O.L. Mangasarian and J.-S. Pang, eds. (Dordrecht: Kluwer) pp. 233-251.[10]